Alliance FTP API and Command Guide

Page Information

Last updated: 09.01.2016

Table of Contents

Chapter 1: Introduction

Alliance FTP Manager contains a number of commands, callable APIs, and bindable APIs to help you extend your user applications to use secure transfer functions and related technologies. This document provides documentation on those APIs.

FTP Manager FTP Commands

FTP Manager provides two types of FTP commands – FTP session commands and FTP functional commands. The session commands perform one FTP action such as sending a file, or retrieving a file. Generally you make one call to an FTP session command and it performs all of the FTP operations that you need. The FTP functional commands let you control each step of an FTP session such as the opening the connection, sending the user name, sending the password, retrieving a file, and so forth. You can do these functions from any CL program and control the flow of the FTP session depending on the results of an operation.

FTP Manager commands can be used from the command line, from CL programs, or anywhere in your applications that you want to execute FTP Manager functions using a command interface.

All commands have full Help available be pressing ***F1*** or the ***Help*** key when prompting a command.

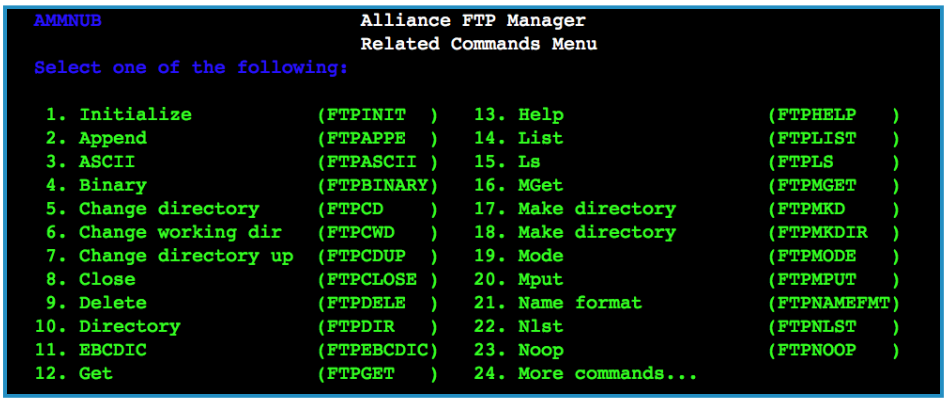


Select option `1` to view the list of functional **`FTP commands`**. Select option `2` to view the **`FTP session commands`**.

FTP Manager FTP Functional Commands

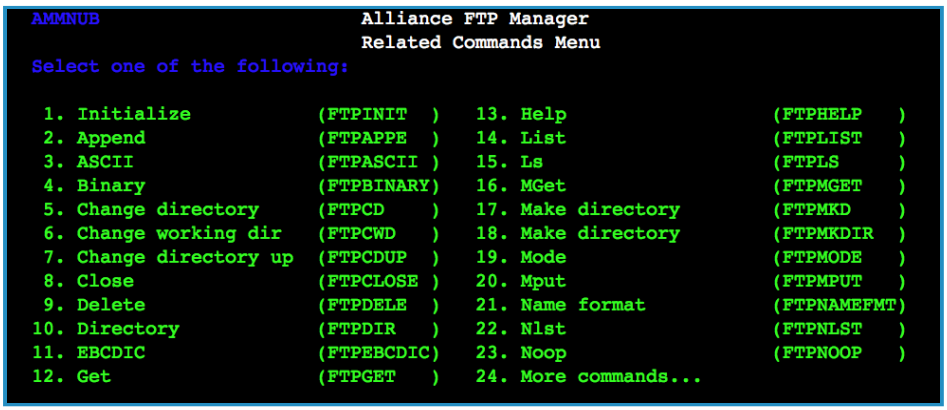
This menu shows all of the FTP Manager functional FTP commands.

These commands are documented in the Alliance FTP Command Reference. Please consult that manual for information on how to use the commands in your own applications.



Select a command to use. Selection option `24` to display the next panel of commands.

The following menu is the next set of FTP Manager commands.



Select a command to use. Selection option `24` to display the next 2 panels of commands.

Enter an option to prompt a command for execution. Note that these commands are only valid in CL programs. The commands return data that let you inspect the results of an FTP function.

Chapter 2: Encryption APIs

Overview

The FTP Manager encryption API provides for secure AES encryption with integrated key management. You can use the following APIs in your RPG and Cobol programs to secure sensitive data such as credit card numbers, social security numbers, etc.

`AESEncCTR2L` – Encrypt with Counter mode

Overview

The `AESEncCTR2` function is used to encrypt data using the Counter (CTR) mode of AES encryption. The AES mode of encryption has the desirable characteristic of not requiring the expansion of the size of a field. For IBM i customers who what to minimize the impact of implementing field level encryption, the Counter mode of encryption is very helpful.

C Function Declaration:

```
void AESEncCTR2L(UCHAR* pcaDefinition,
        ULONG* pulKeyType,
        UCHAR* pcaPass,
        UCHAR* pcaKeyInstance,
        UCHAR* pcaInput,
                ULONG* pulInputLen,
        UCHAR* pcaCTR,
        ULONG* pulCTROption,
        UCHAR* pcaOutput,
        UCHAR* pcaKeyInstanceUsed,
                UCHAR* pcaNextCTR,
        UCHAR* pcaKeyExpiration,
        ULONG* pulReturn);
```

ILE RPG call format:

```
d Definition      s              11a
d KeyType         s               8b 0
d Pass            s              33a
d KeyInstance     s              21a
d InpData         s             256a
d InpLength       s               8b 0
d Counter         s              16a
d CounterOpt      s               8b 0
d OutData         s             256a
d KeyInstUsed     s              21a
d NextCTR         s              16a
d KeyExpires      s               9a
d ReplyCode       s               8b 0

Callb(d)   'AESEncCTR2L'
Parm                    Definition
Parm                    KeyType
Parm                    Pass
Parm                    KeyInstance
Parm                    InpData
Parm                    InpLength
Parm                    Counter
Parm                    CounterOpt
Parm                    OutData
Parm                    KeyInstUsed
Parm                    NextCTR
Parm                    KeyExpires
Parm                    ReplyCode
```

Function Description

This function encrypts IBM i data using the AES Counter mode of encryption.

Definition (Character, 11, null terminated)

The name of the key to use for the encryption request. This must be a valid key in the Alliance AES key file. The field must be null terminated (a null is a single character with the hex value of “00”). This field is case sensitive. You may leave this field blank if you want to use a pass phrase in the following parameters.

Key type (Numeric binary, 4 byte long value)

Enter the value of `1` to use a normal 256-bit key stored in the Alliance key store file. This value used the Pass phrase based encryption (PBE) mode of key generation. Enter the value of `2` to use a raw pass phrase stored in the Alliance key store. The raw pass phrase is loaded as the 256-bit encryption key.

Pass phrase (Character, 129, null terminated)

If you are not providing a key name for encryption, specify a pass phrase. The pass phrase will be treated as the 256-bit raw key. The field must be null terminated. Note that it is considered poor practice to expose encryption key material in your application programs. If you use this field be sure to provide appropriate steps to protect the pass phrase from loss.

Key instance (Character, 21)

The name of the key instance to use for the encryption. This is a null terminated field. If this field is not provided the default instance is used for encryption. This is the recommended value. The actual key instance used will be returned. Generally you should not provide this value so that the default instance of the key is used.

Input data (Character, up to 4294967295)

This field contains the data you want to encrypt. You can define the variable with any length that is a multiple of 16 bytes. In the next field you define the length of the data to encrypt. For example, you can define this field as a 256 byte character field and move a 16 character credit card number into the field. In the field length specify a value of `16`.

Input length (Numeric binary, 4 byte long value)

This field contains the length of the data to be encrypted. The value must be a multiple of 16, however you can encrypt any length value. The resulting encrypted data can be truncated to the length you want.

Counter (Character, 16)

This field contains the Counter value to use for encryption. The counter is an initialization vector designed to insure that the encrypted value is not predictable. With Counter mode encryption you must use a unique value each time you encrypt with a particular key. You can have Alliance assign a unique counter value, but you must store it for the decryption operation. Alternatively, if you have a unique value in the database record, such as a unique customer number, you can use that.

Counter option (Numeric binary, 4 byte long value)

Use the value `0` to indicate that you do not want Alliance to assign a unique counter value. Use the value of `1` to indicate that you want Alliance to assign a unique number. The value assigned will be returned to your program.

Output data (Character, up to 4294967295)

This field contains the encrypted or other output data. This field can be any length but must be large enough to receive the encrypted, hashed, or encoded data. If you encrypted the data and the length is not a multiple of 16 bytes, you can truncate the unused portion of the field.

Key instance used (Character, 20)

This field contains the name of the key instance used. The value is 20 characters long. If you are using key roll-over, you should save this value in your database record in order to decrypt the value using the correct key and instance.

Next counter (Character, 16)

This field contains the next counter value. When Alliance encrypts a block of data it calculates the next counter value. If the length of the encrypted data is 16 bytes, this field will contain the original counter plus one. If the length of the data was 32 bytes, this field will contain the original counter plus 2.

Key expiration date (Character, 8, CCYYMMDD)

If the encryption key has an expiration date, this field contains the date in CCYYMMDD format. If the key is non-expiring this field will contain zeros.

Reply code (Numeric binary, 4 byte long value)

This field contains a reply code indicating the success or failure of the encryption request. The value of `0` (zero) indicates success. Any non-zero value indicates a failure in the request. You should always code your applications to handle an error condition.

Security notes

After completing the encryption or decryption tasks you should always clear the input and output variables used by the API.

Sample code

The sample program **`ENCCTR03`** in source file **`QRPGLESRC`** in the product library provides examples of using this API.

`AESDecCTR2L` – Decrypt with Counter mode

Overview

The **`AESDecCTR2L`** procedure is used to decrypt data using the Counter (CTR) mode of AES encryption. The AES Counter mode of encryption has the desirable characteristic of not requiring the expansion of the size of a field. For IBM i customers who what to minimize the impact of implementing field level encryption, the Counter mode of encryption is very helpful.

C Function Declaration:

```
void AESDecCTR2L(UCHAR* pcaDefinition,
                 ULONG* pulKeyType,
        UCHAR* pcaPass,
        UCHAR* pcaKeyInstance,
        UCHAR* pcaInput,
        ULONG* pulInputLen,
        UCHAR* pcaCTR,
        UCHAR* pcaOutput,
        UCHAR* pcaKeyInstanceUsed,
        UCHAR* pcaKeyExpiration,
        ULONG* pulReturn);
```

ILE RPG call format:

```
d Definition      s              11a
d KeyType         s               8b 0
d Pass            s              33a
d KeyInstance     s              21a
d InpData         s             256a
d InpLength       s               8b 0
d Counter         s              16a
d OutData         s             256a
d KeyInstUsed     s              21a

d KeyExpires      s               9a
d ReplyCode       s               8b 0


Callb(d)   'AESDecCTR2L'
Parm                    Definition
Parm                    KeyType
Parm                    Pass
Parm                    KeyInstance
Parm                    InpData
Parm                    InpLength
Parm                    Counter
Parm                    OutData
Parm                    KeyInstUsed
Parm                    KeyExpires
Parm                    ReplyCode
```

Function Description

This function decrypts IBM i data using the AES Counter mode of encryption.

Definition (Character, 11, null terminated)

The name of the key to use for the decryption request. This must be a valid key in the Alliance AES key file. The field must be null terminated (a null is a single character with the hex value of “00”). This field is case sensitive. You may leave this field blank if you want to use a pass phrase in the following parameters.

Key type (Numeric binary, 4 byte long value)

Enter the value of `1` to use a normal 256-bit key stored in the Alliance key store file. This value used the Pass phrase based encryption (PBE) mode of key generation. Enter the value of `2` to use a raw pass phrase stored in the Alliance key store. The raw pass phrase is loaded as the raw 256-bit encryption key.

Pass phrase (Character, 129, null terminated)

If you are not providing a key name for decryption, specify a pass phrase. The pass phrase will be treated as the 256-bit raw key. The field must be null terminated. Note that it is considered poor practice to expose encryption key material in your application programs. If you use this field be sure to provide appropriate steps to protect the pass phrase from loss.

Key instance (Character, 21)

The name of the key instance to use for the decryption. This is a null terminated field. If this field is not provided the default instance for the key is used for decryption. The actual key instance used will be returned.

Input data (Character, up to 4294967295)

This field contains the data you want to decrypt. You can define the variable with any length that is a multiple of 16 bytes. In the next field you define the length of the data to decrypt. For example, you can define this field as a 256 byte character field and move a 16 character credit card number into the field. In the field length specify a value of `16`.

Input length (Numeric binary, 4 byte long value)

This field contains the length of the data to be decrypted. The value must be a multiple of 16, however you can decrypt any length value. The resulting decrypted data can be truncated to the length you want.

Counter (Character, 16)

This field contains the Counter value to use for encryption. The counter is an initialization vector designed to insure that the encrypted value is not predictable. With Counter mode encryption you must use a value that is unique value for the key. You can have Alliance assign a unique counter value, but you must store it for the decryption operation. Alternatively, if you have a unique value in the database record, such as a unique customer number, you can use that.

Output data (Character, up to 4294967295)

This field contains the decrypted or masked output data. This field can be any length but must be large enough to receive the encrypted, hashed, or masked data. If the encrypted data has a the length that is not a multiple of 16 bytes, you can truncate the unused portion of the field.

Key instance used (Character, 20)

This field contains the name of the key instance used. The value is 20 characters long.

Key expiration date (Character, 8, CCYYMMDD)

If the encryption key has an expiration date, this field contains the date in CCYYMMDD format. If the key is non-expiring this field will contain zeros.

Reply code (Numeric binary, 4 byte long value)

This field contains a reply code indicating the success or failure of the decryption request. The value of `0` (zero) indicates success. Any non-zero value indicates a failure in the request. You should always code your applications to handle an error condition.

Security notes

After completing the encryption or decryption tasks you should always clear the input and output variables used by the API.

Sample code

The sample program **`DECCTR03`** in source file **`QRPGLESRC`** in the product library provides examples of using this API.

Chapter 3: Application Commands

Overview

FTP Manager provides commands that you can use in your CL programs to perform FTP transfers and other FTP related tasks. You can use FTP Manager commands to execute stored FTP definitions and scripts, or you can use FTP Manager commands to execute undefined FTP sessions “on the fly”. All FTP Manager commands issue trappable exception messages if the command does not end in success. The following section defines each command and its parameters.

`FTMPING` – Ping a remote system

Description

The **`FTMPING`** command verifies that your IBM i can communicate with a remote system using the standard **`Ping`** command. The **`FTMPING`** command issues an exception if the remote system does not respond. This command can be helpful in situations where you want to verify that a remote system can be reached before attempting an FTP session.

> ***NOTE:*** Alliance FTP Manager can automatically verify a remote system. Specify that you want to verify the remote system on the FTP Definition for the session.

Command format

You can use the **`FTMPING`** command like this:

```
Ftmping rmtsys(system-name | *intnetadr) intnetadr(ip-address)
```

Parameters

Remote system (***`RMTSYS`*** *), Character, 64*

The domain name of the remote system. If you know the IP address of the remote system but not the domain name, enter the special value **`*intnetadr`** and the IP address in the next field.

IP address (***`INTNETADR`*** *), Character, 15*

The IP address of the remote system you are trying to ping. To use the IP address you must enter **`*intnetadr`** in the previous field and a valid IP address in this field.

Sample program

The sample program **`SAMPING`** in file **`QCLSRC`** in the product library shows how to use this command.

`FTMEXEC` – Execute an FTP session

Description

The **FTMEXEC** command executes an FTP session using the session definition, local IBM i database file, and remote file name that you specify. If there is an error in the session an exception condition is raised.

Command format

You can use the **FTMEXEC** command like this:

```
Ftmexec ftpdfn(ftp-definition) lclfil(local-file-name)
        Rmtfil(remote-file-name)
```

Parameters

*FTP definition (**FTPDFN**) Character, 10*

The name of the FTP Manager definition you want to use for this session. The definition contains the parameters to use for the FTP session including user ID, password, remote system name or IP address, compression options and file backup options. The FTP definition must exist before the FTP session.

*Local file name (**LCLFIL**) Character, 64*

The name of the IBM i file you want to transfer. The name must be specified in FTP format: **LIBRARY/FILE.MEMBER** . For example, if your file is named **ORDERS** in library **PROD** and has a member named **JANUARY** you would specify this parameter as follows:

```
PROD/ORDERS.JANUARY
```

*Remote file name (**RMTFIL**) Character, 64*

The name of the file on the remote system. This can be any name that is compatible with the remote file system.

Sample program

The sample program **SAMPEXEC** in file **QCLSRC** in the product library shows how to use this command.

FTPGETFILE – Retrieve file with SSL FTP

Description

The **FTPGETFILE** command is used to retrieve a file from a remote FTP server using the SSL FTP protocol. The command provides a number of parameters to allow maximum flexibility for the session. For more complex file retrieval operations you can use the Alliance FTP commands which allow for very tight control over every step of the FTP session. In addition to Alliance FTP commands you can use the Alliance bindable FTP APIs in service program FTPCLN.

Command format

You can use the **FTPGETFILE** command like this:

```
FTPGETFILE
        RMTFILE( remote-file-name)
        LCLFILE(local-file-name)
        FILETYPE( *library | *ifs)
        LIBRARY( library-name )
        DIRECTORY( directory-name )
        FTPHOST( host-name or ip address )
        PORT( 21 | port-number)
        FTPUSER( user-name )
        FTPPASS( password )
        ACCOUNT( account-name )
        LOGLEVEL( *low | *medium | *high )
        SSLAPPID( ssl-application-ID )
        CHGDIR ( directory-name )
        CCCOPT( *yes | *no )
        SITE1( site-command-1 )
        SITE2( site-command-2 )
        SITE3( site-command-3 )
        DLTOPT( *yes | *no )
        IMPLICIT( *yes | *no )
        FIREWALL( *yes | *no )
        ALWSSLV2( *yes | *no )
        ALWSSLV3( *yes | *no )
        ALWTLSV1( *yes | *no )
```

Parameters

*Remote file (**RMTFILE**) Character, 128*

The name of the remote file that you want to retrieve. Note that some FTP servers use case-sensitive names. Use single quotes around the file name to preserve the case.

*Local file (**LCLFILE**) Character, 128*

The name of the local file. If the file type is ***IFS** for an IFS file, this should be the name of the IFS file. If the file type is ***LIBRARY** this should be the upper case file name. The library is defined in a following parameter.

*File type (**FILETYPE**) Character, 10*

To receive a file to an IFS directory, use the special value ***IFS** . To receive a file to a library use the special value ***LIBRARY** .

*Library (**LIBRARY**) Character, 10*

If you specified ***LIBRARY** for the file type enter the name of the library to receive the file. The name should be upper case and the library must exist.

*Directory (**DIRECTORY**) Character, 128*

If you specified ***IFS** for the file type enter the full directory path to receive the file. The full file path should start with a slash (**/**) and end without a slash.

*FTP host (**FTPHOST**) Character, 32*

Enter the DNS name or IP address of the remote FTP server. If you use a DNS name you must configure IBM i TCP with the DNS server name.

*Port number (**PORT**) Numeric, range 1 to 65535*

Enter the port number for the connection to the remote FTP server. The default is 21. You can specify any valid port number.

*FTP user name (**FTPUSER**) Character, 32*

Enter the user name for the logon to the remote FTP server.

*FTP password (**FTPPASS**) Character, 32*

Enter the password for the remote FTP server. This is a non-display field. This is a case sensitive field. Use single quotes to preserve the case of the password.

*Account (**ACCOUNT**) Character, 32*

If the remote FTP server requires an account, enter the account name in this field. Some older FTP servers required an account name.

*Logging level (**LOGLEVEL**) Character, 10*

Use the special value ***LOW** for a low level of logging. Use the special value ***MEDIUM** for a moderate level of logging. Use the special value ***HIGH** for a verbose level of logging. When ***HIGH** is used the certificate information is extracted and placed in the application log.

*SSL application ID (**SSLAPPID**) Character, 32*

Enter the SSL application ID that you created with Digital Certificate Manager. Consult the Alliance documentation on how to create a certificate and application ID.

*Change directory (**CHGDIR**) Character, 32*

If you want to change to a different directory after connecting to the remote FTP server, enter the directory in this parameter.

*Clear Command Channel option (**CCCOPT**) Character, 4*

Enter ***YES** to clear the command channel after authenticating with the user name and password. Enter ***NO** to not clear the command channel. Clearing the command channel may be necessary with certain firewall configurations. Note that some FTP servers do not support the Clear Command Channel function.

*Site command 1 (**SITE1**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the first site command in this field.

*Site command 2 (**SITE2**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the second site command in this field.

*Site command 3 (**SITE3**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the third site command in this field.

*Delete file option (**DLTOPT**) Character, 4*

Enter ***YES** to delete the file from the FTP server after retrieving it. Enter ***NO** to leave the file on the FTP server after retrieving it.

*Implicit SSL FTP option (**IMPLICIT**) Character, 4*

Enter ***YES** if you want to use implicit SSL FTP. Enter ***NO** if you do not want to use implicit SSL FTP. Older FTP servers may use implicit SSL FTP sessions. You can communicate with them by enabling this option.

*Intelligent firewall negotiation option (**FIREWALL**) Character, 4*

Enter ***YES** if you want to use intelligent firewall negotiation. Enter ***NO** to not use intelligent firewall negotiation. Intelligent firewall negotiation will ignore the response to a **PORT** command on a file transfer and use the original IP address instead.

*Allow SSL version 2 (**ALWSSLV2**) Character, 4*

Enter ***YES** to enable use of SSL version 2 in the connection to the remote FTP server. Enter ***NO** to not allow use of SSL version 2. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

*Allow SSL version 3 (**ALWSSLV3**) Character, 4*

Enter ***YES** to enable use of SSL version 3 in the connection to the remote FTP server. Enter ***NO** to not allow use of SSL version 3. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

*Allow TLS version 1 (**ALWTLSV1**) Character, 4*

Enter ***YES** to enable use of TLS version 1 in the connection to the remote FTP server. Enter ***NO** to not allow use of TLS version 1. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

Monitoring for messages

If the FTP session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance FTP messages start with the prefix **ATC** . You can globally monitor for errors using message IF **ATC0000** .

Sample program

The sample program **SAMFTPGETF** in file **QCLSRC** in the product library shows how to use this command.

FTPPUTFILE – Send file with SSL FTP

Description

The **FTPPUTFILE** command is used to send a file to a remote FTP server using the SSL FTP protocol. The command provides a number of parameters to allow maximum flexibility for the session. For more complex file transmission operations you can use the Alliance FTP commands which allow for very tight control over every step of the FTP session. In addition to Alliance FTP commands you can use the Alliance bindable FTP APIs in service program **FTPCLN** .

Command format

You can use the **FTPPUTFILE** command like this:

```
FTPPUTFILE
        FILE( library/file/member )
        IFSFILE( local-file-name )
        RMTFILE( remote-file-name )
        FTPHOST( host-name or ip address )
        PORT( 21 | port-number)
        FTPUSER( user-name )
        FTPPASS( password )
        ACCOUNT( account-name )
        LOGLEVEL( *low | *medium | *high )
        SSLAPPID( ssl-application-ID )
        CHGDIR ( directory-name )
        CCCOPT( *yes | *no )
        SITE1( site-command-1 )
        SITE2( site-command-2 )
        SITE3( site-command-3 )
        BINARY( *yes | *no )
        IMPLICIT( *yes | *no )
        FIREWALL( *yes | *no )
        ALWSSLV2( *yes | *no )
        ALWSSLV3( *yes | *no )
        ALWTLSV1( *yes | *no )
```

Parameters

*File (**FILE**) Character, 30*

Enter the name of the file that you want to send. Enter the library, file and member name. If you want to send an IFS file leave this parameter blank and specify the IFS file name in the following parameter.

*IFS file (**LCLFILE**) Character, 128*

If you want to send an IFS file specify the full path name and file name in this field.

*Remote file (**RMTFILE**) Character, 128*

Enter the name of the file for the remote FTP server.

*FTP host (**FTPHOST**) Character, 32*

Enter the DNS name or IP address of the remote FTP server. If you use a DNS name you must configure IBM i TCP with the DNS server name.

*Port number (**PORT**) Numeric, range 1 to 65535*

Enter the port number for the connection to the remote FTP server. The default is **21** . You can specify any valid port number.

*FTP user name (**FTPUSER**) Character, 32*

Enter the user name for the logon to the remote FTP server.

*FTP password (**FTPPASS**) Character, 32*

Enter the password for the remote FTP server. This is a non-display field. This is a case sensitive field. Use single quotes to preserve the case of the password.

*Account (**ACCOUNT**) Character, 32*

If the remote FTP server requires an account, enter the account name in this field. Some older FTP servers required an account name.

*Logging level (**LOGLEVEL**) Character, 10*

Use the special value ***LOW** for a low level of logging. Use the special value ***MEDIUM** for a moderate level of logging. Use the special value ***HIGH** for a verbose level of logging. When ***HIGH** is used the certificate information is extracted and placed in the application log.

*SSL application ID (**SSLAPPID**) Character, 32*

Enter the SSL application ID that you created with Digital Certificate Manager. Consult the Alliance documentation on how to create a certificate and application ID.

*Change directory (**CHGDIR**) Character, 32*

If you want to change to a different directory after connecting to the remote FTP server, enter the directory in this parameter.

*Clear Command Channel option (**CCCOPT**) Character, 4*

Enter ***YES** to clear the command channel after authenticating with the user name and password. Enter ***NO** to not clear the command channel. Clearing the command channel may be necessary with certain firewall configurations. Note that some FTP servers do not support the Clear Command Channel function.

*Site command 1 (**SITE1**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the first site command in this field.

*Site command 2 (**SITE2**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the second site command in this field.

*Site command 3 (**SITE3**) Character, 32*

You can send up to three site commands to the remote FTP server. Enter the third site command in this field.

*Binary option (**DLTOPT**) Character, 4*

Enter ***YES** to transfer the file in binary mode. Enter ***NO** to transfer the file in text mode.

*Implicit SSL FTP option (**IMPLICIT**) Character, 4*

Enter ***YES** if you want to use implicit SSL FTP. Enter ***NO** if you do not want to use implicit SSL FTP. Older FTP servers may use implicit SSL FTP sessions. You can communicate with them by enabling this option.

*Intelligent firewall negotiation option (**FIREWALL**) Character, 4*

Enter ***YES** if you want to use intelligent firewall negotiation. Enter ***NO** to not use intelligent firewall negotiation. Intelligent firewall negotiation will ignore the response to a PORT command on a file transfer and use the original IP address instead.

*Allow SSL version 2 (**ALWSSLV2**) Character, 4*

Enter ***YES** to enable use of SSL version 2 in the connection to the remote FTP server. Enter ***NO** to not allow use of SSL version 2. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

*Allow SSL version 3 (**ALWSSLV3**) Character, 4*

Enter ***YES** to enable use of SSL version 3 in the connection to the remote FTP server. Enter ***NO** to not allow use of SSL version 3. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

*Allow TLS version 1 (**ALWTLSV1**) Character, 4*

Enter ***YES** to enable use of TLS version 1 in the connection to the remote FTP server. Enter ***NO** to not allow use of TLS version 1. At least one of SSL version 2, SSL version 3, or TLS version 1 must be enabled.

Monitoring for messages

If the FTP session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance FTP message start with the prefix **ATC** . You can globally monitor for errors using message IF **ATC0000** .

Sample program

The sample program **SAMFTPGETF** in file **QCLSRC** in the product library shows how to use this command.

FTMAUTO – Scan an FTP or SSH sFTP server

Description

The **FTMAUTO** command is used to scan a remote FTP or SSH sFTP server and pull all files from a specified directory to the local IBM I server. This command works in the same fashion as the Alliance Auto-Scan facility, but runs under your control and only runs one time.

Command format

You can use the **FTMAUTO** command like this:

```
FTMAUTO
SCAN( Auto-scan-definition )
```

Parameters

*Auto-scan definition (**SCAN**), Character, 10*

Enter the name of an auto-scan definition. The auto-scan definition can be inactive. This definition will be used for the information about the FTP or SSH sFTP server.

Monitoring for messages

If the scan session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance messages start with the prefix **ATC** . You can globally monitor for errors using message ID **ATC0000** .

Sample program

The sample program **SAMPAUTO** in file **QCLSRC** in the product library shows how to use this command.

FTMSFTP – Put file with SSH sFTP

Description

The **FTMSFTP** command is used to send one file from an IBM I library to a Secure Shell FTP server (SSH sFTP). The command uses an FTP definition for the server information (IP address or domain name, port number, user name, account, etc.).

Command format

You can use the **FTMSFTP** command like this:

```
FTMSFTP
        SESSION( FTP-definition )
        FILE(  member/library/file )
        FILEEXT( optional-file-extension )
```

Parameters

*FTP definition (**SESSION**), Character, 10*

Enter the FTP definition name you want to use for this SSH sFTP session. You must create this definition name prior to using the command.

*File (**FILE**), Character, 30*

Enter the file, library, and member name to be transmitted to the SSH sFTP server.

*File extension (**FILEEXT**), Charater, 10*

You can specify an optional file extension to be added to the file name. For example, if the file name is **ORDERS** , you can specify the extension as **.txt** and the file will be transmitted as **ORDERS.txt** .

Monitoring for messages

If the SSH sFTP session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance messages start with the prefix **ATC** . You can globally monitor for errors using message ID **ATC0000** .

Sample program

The sample program **SAMPSFTP1** in file **QCLSRC** in the product library shows how to use this command.

FTMSFTPGET – Get file with SSH sFTP

Description

The **FTMSFTPGET** command is used to retrieve one file from a Secure Shell (SSH) FTP server using the Secure FTP (sFTP) protocol. The command uses an FTP definition for the server information (IP address or domain name, port number, user name, account, etc.).

Command format

You can use the **FTMSFTPGET** command like this:

```
FTMSFTPGET
        SESSION( FTP-definition )
        RMTFILE( Remote-file-name )
        LCLFILE( Local-file-name )
        RMTDIR( Remote-directory )
        LCLDIR( Local-directory )
```

Parameters

*FTP definition (**SESSION**), Character, 10*

Enter the FTP definition name you want to use for this SSH sFTP session. You must create this definition name prior to using the command.

*Remote file (**RMTFILE**), Character, 64*

Enter the name of the remote file that you want to retrieve. This should be only the file name and not the full path to the file.

*Local file (**LCLFILE**), Character, 64*

Enter the name for the local file. Files are retrieved to an IFS directory on your IBM I platform.

*Remote directory (**RMTDIR**), Character, 64*

Enter the remote directory name, or leave blank. If a value is provided for this field the sFTP client will issue a Change Directory (**CD**) command to the directory before retrieving the file.

*Local Directory (**LCLDIR**), Character, 64*

Enter the local IFS directory name where you want to retrieve the file. If you leave this field blank the current local directory will be used.

Monitoring for messages

If the SSH sFTP session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance messages start with the prefix **ATC** . You can globally monitor for errors using message ID **ATC0000** .

Sample program

The sample program **SAMPSFTP2** in file **QCLSRC** in the product library shows how to use this command.

FTMZIP – Zip compress and encrypt

Description

The **FTMZIP** command provides the ability to zip compress one or more files in an IFS directory with the option to encrypt the files with 256-bit AES encryption. The command supports encrypting one or more files, wild card selection, and encryption of sub-directories. You can retain or discard the path information for the zip archive.

The use of AES encryption by this command follows the WinZip standard. At the time this documentation was written encrypted archives could be decrypted by WinZip, PKZip, and applications on Mac and other platforms. Because there is no independent standard for zip encryption you should test encrypted archives with your target users before deploying this option.

> ***NOTE:*** Alliance FTP Manager includes a Windows command line application to encrypt and decrypt files that is compatible with this command. Your license to Alliance FTP Manager allows you to distribute this application.

Command format

You can use the **FTMZIP** command like this:

```
FTMZIP
        IFSFILE( full-path )
        ZIPFILE( full-path )
        REPLACE( *yes | *no )
        ENCRYPT( *yes | *no )
        PASSWORD( password )
        NAMEOPT(*yes | *no )
        SUBTREE( *none | *all )
        DIROPT( *yes | *no)
        VERBOSE( *yes | *no )
        JAVAVER( '1.5.1' | version)
```

Parameters

*IFS file (**IFSFILE**) Character, 128*

The IFS file you want to zip compress. You can use the wildcard character "*" to select multiple files. You can also use the wildcard character "?" to select any character in the respective position in file names. If you want to encrypt all files in a directory and sub-directory use the directory name for this field. For example, "/directory/mydirectory".

*ZIP file (**ZIPFILE**) Character, 128*

The name of the zip file. If you do not specify the **.zip** extension it will be added automatically. If the Zip file exists you can specify the replace option in the next parameters.

*Replace (**REPLACE**), Character, 10*

Specify ***YES** to replace an existing zip file. Specify ***NO** to not replace the file. The default is ***NO** . If you specify No for this parameter and the zip file exists the command will fail.

*Encrypt (**ENCRYPT**), Character, 10*

Specify ***YES** to encrypt the contents of the zip file. Specify ***NO** to create a non-encrypted Zip file. The default is ***NO** . If you specify Yes for this parameter you must specify a password in the next parameter.

*Password (**PASSWORD**), Character, 32*

If you specified Yes for the encryption option enter a password in this field. You should use a strong password for proper encryption security.

*Name option (**NAMEOPT**), Character, 10*

Specify ***YES** to include the full directory path with the file name. Specify ***NO** to use only the file name in the archive. The default is ***YES** to include the full path in the file name.

*Subtree (**SUBTREE**), Character, 10*

Specify ***ALL** to include sub-directories in the zip archive. Specify ***NONE** to include only the input directory in the archive. The default is ***NONE** . If you specify ***ALL** the files in the sub-directories will be included in the archive. If you've specified wildcard characters in the input file name, these wildcard selections will be applied to the sub-directories.

*Include directories (**DIROPT**), Character, 10*

Specify ***YES** to include an entry for each directory. Specify ***NO** to not include specific directory entries in the archive. The default is ***NO** . If you specify ***YES** empty directories will be included in the zip archve.

*Verbose (**VERBOSE**), Character, 10*

Specify ***YES** to create a verbose output log for the zip session. Specify ***NO** to create no output log. The default is ***NO** . If you specify ***YES** the log will be printed to the **QPRINT** spooled file in a separate Java command processing thread. You can use the Work With Spooled Files (**WRKSPLF**) command to locate the log.

*Java version (**JAVAVER**), Character, 10*

Specify the Java version for this job. The default is "1.5.1", which is the earliest version of Java that is supported. You can specify a later version of Java if you wish.

Monitoring for messages

If the Zip session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance Zip messages start with the prefix **ATC** . You can globally monitor for errors using message **ATC0000** .

Sample program

The sample program **SAMZIPENC** in file **QCLSRC** in the product library shows how to use this command.

FTMUNZIP – Unzip and decrypt

Description

The **FTMUNZIP** command provides the ability to uncompress a zip archive in an IFS directory with the option to decrypt the files with 256-bit AES encryption.

The use of AES deccryption by this command follows the WinZip standard. At the time this documentation was written decrypted archives from WinZip are supported. PKZip encryption is not compatible. Because there is no independent standard for zip encryption you should test encrypted archives before deploying this option.

> ***NOTE:*** Alliance FTP Manager includes a Windows command line application to encrypt and decrypt files that is compatible with this command. Your license to Alliance FTP Manager allows you to distribute this application.

Command format

You can use the **FTMUNZIP** command like this:

```
FTMUNZIP
        ZIPFILE( full-path )
        OUTDIR( *SAME | directory )
        DECRYPT( *yes | *no )
        PASSWORD( password )
        PROMPT(*yes | *no )
        OVROPT( *yes | *no  )
        IGNDIR( *yes | *no)
        VERBOSE( *yes | *no )
        JAVAVER( '1.5.1' | version)
```

Parameters

*ZIP file (**ZIPFILE**) Character, 128*

The name of the input zip file you wish to uncompress.

*Output directory (**OUTDIR**), Character, 128*

Use the value ***SAME** to uncompress to the same directory as the input zip file. Or, you can specify a directory name.

*Decrypt (**DECRYPT**), Character, 10*

Specify ***YES** to decrypt the contents of the zip file. Specify ***NO** to uncompress a non-encrypted zip archive. The default is ***NO** .

*Password (**PASSWORD**), Character, 32*

If you specified Yes for the decryption option enter a password in this field.

*Prompt option (**PROMPT**), Character, 10*

Specify ***YES** to allow prompting for input parameters. Specify ***NO** to not allow user input. The default is ***NO** . Prompted input is not possible in batch jobs and you should always specify ***NO** for this environment.

*Overwrite files (**OVROPT**), Character, 10*

Specify ***YES** to overwrite any existing files during the uncompress operation. Specify ***NO** to not overwrite existing files. The default is ***NO** .

*Ignore directories (**IGNDIR**), Character, 10*

Specify ***YES** to ignore the directory in the file name in the archive. With this option files will be extracted to the output directory. Specify ***NO** to use the full path name of the uncompressed files. Directories will be created if they do not exist. The default is ***NO** .

*Verbose (**VERBOSE**), Character, 10*

Specify ***YES** to create a verbose output log for the zip session. Specify ***NO** to create no output log. The default is ***NO** . If you specify ***YES** the log will be printed to the **QPRINT** spooled file in a separate Java command processing thread. You can use the Work With Spooled Files (**WRKSPLF**) command to locate the log.

*Java version (**JAVAVER**), Character, 10*

Specify the Java version for this job. The default is “1.5.1”, which is the earliest version of Java that is supported. You can specify a later version of Java if you wish.

Monitoring for messages

If the Zip session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance Zip messages start with the prefix **ATC** . You can globally monitor for errors using message **ATC0000** .

Sample program

The sample program **SAMUNZIPE** in file **QCLSRC** in the product library shows how to use this command.

FTMPDF – Create encrypted PDF file

Description

The **FTMPDF** command creates an output PDF file from an input command file with the option to encrypt the output file with 256-bit AES encryption.

> ***NOTE:*** Please see the Alliance PDF Guide for a more detailed description of the input options for this command.

Command format

You can use the **FTMPDF** command like this:

```
FTMPDF

         CTLFILE ( full-path )
         PDFFILE ( full-path )
         REPLACE ( *yes | *no )
         ENCRYPT ( *yes | *no )
         USERPASS( user-password )
         OWNERPASS( owner-password )
         PAGEWIDTH( 612 | page-width )
         PAGEHEIGHT( 792 | page-height )
         LEFT ( 72 | left-margin )
         RIGHT( 72 | righ-margin )
         TOP( 72 | top-margin )
         BOTTOM( 72 | bottom-margin )
         VERBOSE( *yes | *no )
         JAVAVER( '1.5.1' | java-version )
```

Parameters

*Control file (**CTLFILE**), Character, 128*

The name of the input control file. The control file contains directives related to font size, color, headers, footers, watermarks, and other formats. It also contains the text content of the PDF file. See the Alliance PDF Guide for more information.

*PDF file (**PDFFILE**), Character, 128*

The name of the output PDF file.

*Replace (**REPLACE**), Character, 10*

Specify ***YES** to replace the PDF file if it exists. Specify ***NO** to not replace an existing file. The default is ***NO** .

*Encrypt (**ENCRYPT**), Character, 10*

Specify ***YES** to encrypt the PDF file. Specify ***NO** to not encrypt the PDF file. If you specify Yes to encrypt the PDF file the file will be protected with AES encryption and you must specify a user password and an owner password.

*User password (**USERPASS**), Character, 32*

If you specified Yes to encrypt the PDF file enter the user password in this field. You should use a strong password to protect the PDF file.

*Owner password (**OWNERPASS**), Character, 32*

If you specified Yes to encrypt the PDF file enter the owner password in this field. You should use a strong password to protect the PDF file. Note that you can use the same password for the user and the owner. Please see the documentation on the Portable Document Format for information about user and owner permissions.

*Page width (**PAGEWIDTH**), Numeric, 5.0*

Specify the page width in pixels. The default is 612 (8 ½ inches).

*Page height (**PAGEHEIGHT**), Numeric, 5.0*

Specify the page height in pixels. The default is 792 (11 inches).

*Left margin (**LEFT**), Numeric, 5.0*

Specify the left margin in pixels. The default is 72 (1 inch).

*Right margin (**RIGHT**), Numeric, 5.0*

Specify the right margin in pixels. The default is 72 (1 inch).

*Top margin (**TOP**), Numeric, 5.0*

Specify the top margin in pixels. The default is 72 (1 inch).

*Bottom margin (**BOTTOM**), Numeric, 5.0*

Specify the bottom margin in pixels. The default is 72 (1 inch).

*Verbose (**VERBOSE**), Character, 10*

Specify ***YES** to create a verbose output log for the zip session. Specify ***NO** to create no output log. The default is ***NO** . If you specify ***YES** the log will be printed to the **QPRINT** spooled file in a separate Java command processing thread. You can use the Work With Spooled Files (**WRKSPLF**) command to locate the log.

*Java version (**JAVAVER**), Character, 10*

Specify the Java version for this job. The default is “1.5.1”, which is the earliest version of Java that is supported. You can specify a later version of Java if you wish.

Monitoring for messages

If the PDF session is not successful an escape message will be sent to the calling program. You can monitor for an error using the **MONMSG** command. All of the Alliance PDF messages start with the prefix **ATC** . You can globally monitor for errors using message **ATC0000** .

Sample program

The sample program **SAMPDFENC** in file **QCLSRC** in the product library shows how to use this command.

Chapter 4: Application Program Interfaces

Overview

The FTP Manager high level application program interfaces provide an easy way to initiate FTP sessions from your user applications. The APIs can be called from any CL, RPG, or Cobol program as a simple call. They provide return codes that let you know the status of your request and a text description.

FTP client bindable APIs for ILE applications

FTP Manager includes a service program named **FTPCLN** . This service program contains all of the FTP functions (Open, User, Password, Get, Put, etc.). You can extend any of your ILE RPG, Cobol, and CL programs to perform FTP functions by binding your application with this service program. Function prototypes are provided for ILE RPG and C applications. You will find example code in the product library.

Please refer to the Alliance FTP Command Reference for a description of the bindable FTP functions.

API call format

The format for the call to the API is always in the form:

```
call pgm(api-program) parm(&parm1 &parm2 &return &text)
```

The API program will accept one or more input parameters depending on the function it performs. It will return a 2 byte return code and text field that will indicate the success or failure of the API. If the return code is “00” the request was successful. Any other value indicates an error. If there is a failure you can use the information returned in the text field to determine the cause.

DESCHK – Encrypt field for Checkfree using DES

Description

The **DESCHK** API is used to create an encrypted Consolidated Logon field in ASCII hex format that can be used to send User ID and PIN information to Checkfree. If you are only exchanging encrypted files with Checkfree you will not need to use this API. Please consult the Checkfree documentation on Consolidated Logon for information on how to receive and manage DES keys.

Calling format

You can call the API like this:

```
Call pgm(deschk) parm(&key1 &key2 &key3 &iv &data &output &error)
```

Parameters

&key1 (Character, 16)

This is the first DES encryption key in ASCII hex format. You must provide this key to the API.

&key2 (Character, 16)

This is the second DES encryption key in ASCII hex format. You must provide this key to the API.

&key3 (Character, 16)

This is the third DES encryption key in ASCII hex format. You must provide this key to the API.

&iv (Character, 16)

This is the initialization vector in ASCII hex format. You must provide this key to the API.

&data (Character, 48)

The input data you want to encrypt. The data must be in ASCII hex format. Checkfree requires that you blank fill the fields to their specified size.

&output (Character, 48)

This is the encrypted data returned by the API. The data will be in ASCII hex format.

&error (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed.

Sample

The sample program `SAMPCHK` in source file `QCLSRC` in the product library gives an example of how to use this API.

`DESCHKD` – Decrypt field for Checkfree using DES

Description

The `DESCHKD` API is used to decrypt an encrypted Consolidated Logon field in ASCII hex format. It provides the ability to decrypt a field that you encrypted using the `DESCHK` API. Note that you will not need to use the decryption function to work with Checkfree. It is provided as a convenience.

Calling format

You can call the API like this:

```
Call pgm(deschkd) parm(&key1 &key2 &key3 &iv &data &output &error)
```

Parameters

&key1 (Character, 16)

This is the first DES encryption key in ASCII hex format. You must provide this key to the API.

&key2 (Character, 16)

This is the second DES encryption key in ASCII hex format. You must provide this key to the API.

&key3 (Character, 16)

This is the third DES encryption key in ASCII hex format. You must provide this key to the API.

&iv (Character, 16)

This is the initialization vector in ASCII hex format. You must provide this key to the API.

&data (Character, 48)

The input data you want to decrypt. The data must be in ASCII hex format as created by the DESCHK encryption API.

&output (Character, 48)

This is the decrypted data returned by the API. The data will be in ASCII hex format.

&error (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed.

Sample

The sample program `SAMPCHKD` in source file `QCLSRC` in the product library gives an example of how to use this API.

`DESENCR` – Encrypt data using Triple DES

Description

The `DESENCR` API is used to encrypt any data using Triple DES. This API is based on the Triple DES EDE (encrypt-decrypt-encrypt) outer CBC algorithm. You can use this Triple DES API to encrypt data in database files. Unlike the Checkfree APIs, this API takes standard IBM i data as input. It is much faster than PGP encryption functions and is considered secure by Visa and other organizations.

Calling format

You can call the API like this:

```
Call pgm(desencr) parm(&key1 &key2 &key3 &iv &data &output &error)
```

Parameters

&key1 (Character, 16)

This is the first DES encryption key in ASCII hex format. You must provide this key to the API.

&key2 (Character, 16)

This is the second DES encryption key in ASCII hex format. You must provide this key to the API.

&key3 (Character, 16)

This is the third DES encryption key in ASCII hex format. You must provide this key to the API.

&iv (Character, 16)

This is the initialization vector in ASCII hex format. You must provide this key to the API.

&data (Character, 512)

The input data you want to encrypt. The length of the data must not exceed 512 bytes.

&output (Character, 1024)

This is the encrypted data returned by the API. The data will be in hex format.

&error (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed.

Sample

The sample program `SAMPENCR` in source file `QCLSRC` in the product library gives an example of how to use this API.

`DESDECR` – Decrypt data using Triple DES

Description

The `DESDECR` API is used to decrypt any data encrypted using Triple DES. This API is based on the Triple DES EDE (encrypt-decrypt-encrypt) outer CBC algorithm. You can use this Triple DES API to decrypt data in database files. This API is the complement of the `DESENCR` API.

Calling format

You can call the API like this:

```
Call pgm(desdecr) parm(&key1 &key2 &key3 &iv &data &output &error)
```

Parameters

&key1 (Character, 16)

This is the first DES encryption key in ASCII hex format. You must provide this key to the API.

&key2 (Character, 16)

This is the second DES encryption key in ASCII hex format. You must provide this key to the API.

&key3 (Character, 16)

This is the third DES encryption key in ASCII hex format. You must provide this key to the API.

&iv (Character, 16)

This is the initialization vector in ASCII hex format. You must provide this key to the API.

&data (Character, 1024)

The encrypted input data you want to decrypt. The length of the data must not exceed 512 bytes.

&output (Character, 1024)

This is the decrypted data returned by the API.

&error (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed.

Sample

The sample program `SAMPENCR` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMEDICPY` – Copy EDI file to IFS in stream format

Description

The `FTMEDICPY` API provides a way to copy an EDI file in wrapped format to an IFS directory. This API will determine the segment terminator used in the document and compress blanks from the end of each segment and the final IEA segment. It is designed to be used in conjunction with Alliance AS2 Integrator to copy an EDI file to the IFS for transmission to an IBM i or Windows server for transmission over the Internet.

Calling format

You can call the API like this:

```
Call pgm(ftmedicpy) parm(&file &library &member &table
   &tablelib &ifsfile &error)
```

Parameters

&file (Character, 10)

The name of the file. This must be a file containing EDI data in wrapped format.

&library (Character, 10)

The name of the library where the file resides. Use the actual library name and not special values such as `*LIBL` or `*CURLIB` .

&member (Character, 10)

The name of the member in the file. Use the actual member name and not special values such as `*FIRST` .

&table (Character, 10)

If you want to convert the data to the `ASCII` character set specify the table name to use. IBM provides a table named `QASCII` in library `QSYS` , or you can use an Alliance table such as `ANEB2AS` in the product library. Leave this field blank if you do not want to convert the data.

&tablelib (Character, 10)

Specify the library name for the conversion table.

&ifsfile (Character, 255)

Enter the fully qualified name of the IFS file to create. You can use the /tmp directory in the IFS file system if you wish. You should not use the QDLS folder system for stream files.

&error (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMEDICPY` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMEXEC` – Run FTP Session

Description

The `FTMEXEC` API provides a simple call interface to run an FTP session. You pass it the name of an FTP definition, a text field for the API to return a text description of the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined.

Calling format

You can call the API like this:

```
Call pgm(ftmexec) parm(&session &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMEXEC` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMEXECD` – Run FTP Session for IFS file transfer

Description

The **FTMEXECD** API provides a simple call interface to run an FTP session. You pass it the name of an FTP definition, a local IFS file name, a rename file name, a text field for the API to return a text description of the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined.

Calling format

You can call the API like this:

```
Call pgm(ftmexecd) parm(&session &local &remote &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&local (Character, 64)

The name of the local IFS file you want to transfer. This should be a full path to the file including the directory and any sub-directories.

&remote (Character, 64)

The name of the file name on the remote server.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **00** ` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMEXEC** in source file **QCLSRC** in the product library gives an example of how to use this API.

FTMEXECDL – Run FTP Session for IFS file transfer with long name

Description

The **FTMEXECDL** API provides a simple call interface to run an FTP session. You pass it the name of an FTP definition, a local IFS file name, a rename file name, a text field for the API to return a text description of the session, and a return code. The local and remote file names can be up to 256 bytes in length. The session defined by the FTP definition will be executed including any substitutions you have defined.

Calling format

You can call the API like this:

```
Call pgm(ftmexecdl) parm(&session &local &remote &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&local (Character, 256)

The name of the local IFS file you want to transfer. This should be a full path to the file including the directory and any sub-directories.

&remote (Character, 256)

The name of the file name on the remote server.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMEXEC`** in source file **`QCLSRC`** in the product library gives an example of how to use this API.

`FTMEXECF` – Run FTP Session with File Substitution

Description

The **`FTMEXECF`** API provides a simple call interface to run an FTP session and lets you specify a local IBM i file name. You pass it the name of an FTP definition, a file name, a library name containing the file, a text field for the API to return a text description of the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

This API assumes that the member name in the file is the same name as the file. If the member name is different than the file name use API program **`FTMEXECFM`** . Note that the substitution value **`&ll`** (local file name) will be the file name you specify on the parameters passed to this API.

Calling format

You can call the API like this:

```
Call pgm(ftmexecf) parm(&session &file &library &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMEXECF`** in source file **`QCLSRC`** in the product library gives an example of how to use this API.

FTMEXECFM – Run FTP Session with File and Member Substitution

Description

The **`FTMEXECFM`** API provides a simple call interface to run an FTP session and lets you specify a local IBM i file name and the member name. This is similar to the **`FTMEXECF`** API, but provides you the ability to specify the member name. You pass this API the name of an FTP definition, a file name, a library name containing the file, a member name, a text field for the API to return a text description of the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Note that the substitution value &ll in the script file will be the file, library and member name you pass to the API program. If you need to specify the name of the remote file use API program **`FTMEXECI`** .

Calling format

You can call the API like this:

```
Call pgm(ftmexecfm) parm(&session &file &library &member &text &ret
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&member (Character, 10)

The name of the member in the file.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMEXECFM`** in source file **`QCLSRC`** in the product library gives an example of how to use this API.

`FTMEXECFM2` – Run FTP Session with File Name, Force Extension

Description

The **`FTMEXECFM2`** API provides a simple call interface to run an FTP session and lets you specify a local IBM i file name and extension for the remote file. This is similar to the **`FTMEXECFM`** API, but provides you the ability to specify the file extension to use. This is helpful if the file must have a particular extension on the remote system. You pass this API the name of an FTP definition, a file name, a library name containing the file, a member name, the extension option, the actual extension to use, a text field for the API to return a text description of the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Note that the substitution value &ll in the script file will be the file, library and member name you pass to the API program. If you need to specify the name of the remote file use API program **FTMEXECI**.

Calling format

You can call the API like this:

```
Call pgm(ftmexecfm2) parm(&session &file &library &member &extopt &
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&member (Character, 10)

The name of the member in the file.

&extopt (Character, 1)

Specify `1` to force the use of an extension. Specify `0` to not use any extension. If you specify `1` you must provide an extension in the following field. An extension must include the period (`.`) and extension value to use. For example, **`.txt`**.

&extension (Character, 8)

The extension to use. Include the period (`.`) and file extension value.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMEXECFM2** in source file **QCLSRC** in the product library gives an example of how to use this API.

FTMEXECR – Run FTP Session with Local and Remote File Substitution

Description

The **`FTMEXECR`** API provides a simple call interface to run an FTP session and lets you specify both a local IBM i file name and the remote file name. You pass it the name of an FTP definition, a local file and library name, a remote file name, a text field for the API to return a text description of an errors in the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Note that the substitution value **`&ll`** in the script file will be the file and library name you pass to the API program. The substitution value **`&rr`** will be the remote file name you pass to the API.

Calling format

You can call the API like this:

```
Call pgm(ftmexecr) parm(&session &file &library &remote &text &retu
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&remote (Character, 64)

The name of the file on the remote system. This name must be compatible with the file system you are sending the file to.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMEXECR` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMEXECI` – Run FTP Session with Local IFS file and Remote File Substitution

Description

The `FTMEXECI` API provides a simple call interface to run an FTP session and lets you specify both a local IBM i IFS file name and the remote file name. You pass it the name of an FTP definition, a local IFS file name, a remote file name, a text field for the API to return a text description of an errors in the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Calling format

You can call the API like this:

```
Call pgm(ftmexeci) parm(&session &localfile &remote &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&localfile (Character, 64)

The name of the IFS file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file. To send a file in the IFS file system you must specify the fully qualified path. For example if you have a file named “report.txt” in the /user/jsmith IFS directory you would specify this field as:

```
/user/jsmith/report.txt
```

You can send files in the Shared Folder, or QDLS, file system by qualifying the file name with `QDLS` . For example, if you have a file named “report.txt” in a folder named JSMITH you would specify the field as:

```
/QDLS/jsmith/report.txt
```

You can use this API to send files in the QSYS file system, but you might find it easier to use the **FTMEXECR** API instead. To specify a file in the QSYS library system you must fully qualify the name using IFS file naming conventions. For example:

```
/QSYS.LIB/MYLIB.LIB/MYFILE.FILE/MYMEMBER.MBR
```

&remote (Character, 64)

The name of the file on the remote system. This name must be compatible with the file system you are sending the file to.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **00** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMEXECI** in source file **QCLSRC** in the product library gives an example of how to use this API.

FTMEXECL – Run FTP Session with Local IFS file and Remote File Substitution

Description

The **FTMEXECL** API provides a simple call interface to run an FTP session and lets you specify both a local IBM i IFS file name and the remote file name. It is similar to **FTMEXECI** but allows you to specify local and remote file names of up to 255 characters each. You pass the API the name of an FTP definition, a local IFS file name, a remote file name, a text field for the API to return a text description of an errors in the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Calling format

You can call the API like this:

```
Call pgm(ftmexecl) parm(&session &localfile &remote &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be used for this FTP session.

&localfile (Character, 255)

The name of the IfS file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file. To send a file in the IFS file system you must specify the fully qualified path. For example if you have a file named “report.txt” in the /user/jsmith IFS directory you would specify this field as:

```
/user/jsmith/report.txt
```

You can send files in the Shared Folder, or QDLS, file system by qualifying the file name with `QDLS` . For example, if you have a file named “report.txt” in a folder named JSMITH you would specify the field as:

```
/QDLS/jsmith/report.txt
```

You can use this API to send files in the `QSYS` file system, but you might find it easier to use the FTMEXECR API instead. To specify a file in the `QSYS` library system you must fully qualify the name using IFS file naming conventions. For example:

```
/QSYS.LIB/MYLIB.LIB/MYFILE.FILE/MYMEMBER.MBR
```

&remote (Character, 255)

The name of the file on the remote system. This name must be compatible with the file system you are sending the file to.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMEXECL` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMFSTAT` – Change library scan file action status

Description

The **FTMFSTAT** API program lets you dynamically change the status of a library scan file action. You can change the file action from active to inactive, or from inactive to active. This API can be helpful if you need to process files in library scan based on some external event such as a job completing successfully, or the presence of another file.

Calling format

You can call the API like this:

```
Call pgm(ftmfstat) parm(&fileact &library &status &text &return)
```

Parameters

&fileact (Character, 10)

The name of the library scan file action.

&library (Character, 10)

The name of the library scan definition

&status (Character, 1)

The new status for the file action. The value of **1** makes the file action active. The value of **2** makes the file action inactive.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **0000** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMPFSTAT** in source file **QCLSRC** in the product library demonstrates how to call this API.

FTMGXSRCV – Receive files from GE eXchange Services (GXS)

Description

The **FTMGXSRCV** API program lets you receive files from GXS and write them to a library you define. This API accepts the name of an IBM i library to receive to, connects to GXS, logs into your GXS account with your GXS user ID and password, and receives all of the available files in binary mode.

Calling format

You can call the API like this:

```
CALL PGM(FTMGXSRCV) PARM(&LIBRARY &USERID &PASSWORD &LOGOPT
&SSLAPPID &RETRY &RETURN)
```

Parameters

&library (Character, 10)

The name of the library you want to use for receiving files.

&userid (Character, 16)

The GXS user ID. You receive this user ID for the GXS FTP server from GXS.

&password (Character, 16)

The GXS password. You receive the password for the GXS FTP server from GXS.

&logopt (Character, 1)

Use the value of `1` to indicate a low level of logging. Use the value of `2` to indicate a moderate level of logging. Use the value of `3` to indicate a high level of logging. A value of `3` will log all of the data, except passwords, exchanged between the IBM i and the GXS FTP server.

&sslappid (Character, 32)

Enter the SSL application ID to use for the FTP server. The SSL application ID is created by Digital Certificate Manager. Please consult the Alliance Installation and User Guide for information on creating an SSL application ID and assigning a certificate to the SSL connection.

&retry (Character, 2)

Enter the number of retries for receiving files from GXS. It is recommended that you use a value of `05` or higher. Valid values are from `00` to `99`.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMPGXSRCV`** in source file **`QCLSRC`** in the product library demonstrates how to call this API.

`FTMGXSSND` – Send a file to GE eXchange Services (GXS)

Description

The **`FTMGXSSND`** API program lets you send an IBM i database file to GXS. This API accepts the name of an IBM i database file to send, connects to GXS, logs into your GXS account with your GXS user ID and password, and sends the file in binary mode.

Calling format

You can call the API like this:

```
CALL PGM(FTMGXSSND) PARM(&FILE &LIBRARY &MEMBER &USERID +
        &PASSWORD &LOGOPT &SSLAPPID &RETRY &RETURN)
```

Parameters

&file (Character, 10)

The name of the file to send to GXS.

&library (Character, 10)

The name of the library containing the file

&member (Character, 10)

The name of the member in the file you want to send.

&userid (Character, 16)

The GXS user ID. You receive this user ID for the GXS FTP server from GXS.

&password (Character, 16)

The GXS password. You receive the password for the GXS FTP server from GXS.

&logopt (Character, 1)

Use the value of `1` to indicate a low level of logging. Use the value of `2` to indicate a moderate level of logging. Use the value of `3` to indicate a high level of logging. A value of `3` will log all of the data, except passwords, exchanged between the IBM i and the GXS FTP server.

&sslappid (Character, 32)

Enter the SSL application ID to use for the FTP server. The SSL application ID is created by Digital Certificate Manager. Please consult the Alliance Installation and User Guide for information on creating an SSL application ID and assigning a certificate to the SSL connection.

&retry (Character, 2)

Enter the number of retries for sending the file to GXS. It is recommended that you use a value of `05` or higher. Valid values are from `00` to `99`.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMPGXSSND`** in source file **`QCLSRC`** in the product library demonstrates how to call this API.

`FTMIBMRCV` – Receive files from IBM Global Network (Advantis)

Description

The `FTMIBMRCV` API program lets you receive files from IBM and write them to a library you define. This API accepts the name of an IBM i library to receive to, connects to IBM, logs into your IBM account with your IBM user ID and password, and receives all of the available files in binary mode.

Calling format

You can call the API like this:

```
CALL PGM(FTMIBMRCV) PARM(&LIBRARY &USERID &PASSWORD &LOGOPT
&SSLAPPID &RETRY &DIR &RETURN)
```

Parameters

&library (Character, 10)

The name of the library you want to use for receiving files.

&userid (Character, 16)

The IBM account and user ID. You receive this user ID for the IBM FTP server from IBM.

&password (Character, 16)

The IBM password. You receive the password for the IBM FTP server from IBM.

&logopt (Character, 1)

Use the value of `1` to indicate a low level of logging. Use the value of `2` to indicate a moderate level of logging. Use the value of `3` to indicate a high level of logging. A value of `3` will log all of the data, except passwords, exchanged between the IBM i and the IBM FTP server.

&sslappid (Character, 32)

Enter the SSL application ID to use for the FTP server. The SSL application ID is created by Digital Certificate Manager. Please consult the Alliance Installation and User Guide for information on creating an SSL application ID and assigning a certificate to the SSL connection.

&retry (Character, 2)

Enter the number of retries for receiving files from IBM. It is recommended that you use a value of `05` or higher. Valid values are from `00` to `99`.

&dir (Character,3 2)

If you want to change the directory to receive from enter a directory name in this variable.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMPIBMRCV` in source file `QCLSRC` in the product library demonstrates how to call this API.

`FTMIBMSND` – Send a file to IBM Global Network (Advantis)

Description

The `FTMIBMSND` API program lets you send an IBM i database file to IBM. This API accepts the name of an IBM i database file to send, connects to IBM, logs into your IBM account with your IBM user ID and password, and sends the file in binary mode.

Calling format

You can call the API like this:

```
CALL PGM(FTMIBMSND) PARM(&FILE &LIBRARY &MEMBER &USERID +
        &PASSWORD &LOGOPT &SSLAPPID &RETRY &DIR &RETURN)
```

Parameters

&file (Character, 10)

The name of the file to send to IBM.

&library (Character, 10)

The name of the library containing the file

&member (Character, 10)

The name of the member in the file you want to send.

&userid (Character, 16)

The IBM user ID. You receive this user ID for the IBM FTP server from IBM.

&password (Character, 16)

The IBM password. You receive the password for the IBM FTP server from IBM.

&logopt (Character, 1)

Use the value of `1` to indicate a low level of logging. Use the value of `2` to indicate a moderate level of logging. Use the value of `3` to indicate a high level of logging. A value of `3` will log all of the data, except passwords, exchanged between the IBM i and the IBM FTP server.

&sslappid (Character, 32)

Enter the SSL application ID to use for the FTP server. The SSL application ID is created by Digital Certificate Manager. Please consult the Alliance Installation and User Guide for information on creating an SSL application ID and assigning a certificate to the SSL connection.

&retry (Character, 2)

Enter the number of retries for sending the file to IBM. It is recommended that you use a value of `05` or higher. Valid values are from `00` to `99`.

&dir (Character,3 2)

If you want to change the directory to send to enter a directory name in this variable.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMPIBMSND`** in source file **`QCLSRC`** in the product library demonstrates how to call this API.

`FTMIFSTRAN` – Convert an IFS file using a table

Description

The **`FTMIFSTRAN`** API lets you perform IFS character translation on an IFS file. This is helpful if you are using PGP encryption to encrypt a file, but need to transmit the file as an **`EBCDIC`** file. You can use this API to convert any IFS file using any translation table. The API converts the IFS file in place – the original contents of the file are lost. You must back up the file before conversion if you want to keep a copy of the original file.

Calling format

You can call the API like this:

```
Call pgm(ftmifstran) parm(&path &table &library &text &return)
```

Parameters

&path (Character, 128)

The fully qualified IFS path and file name. For example,

```
/tmp/myfile.txt
```

&table (Character, 10)

The name of the table you want to use for the translation. You can use the IBM table **QEBCDIC** in library **QSYS** for **ASCII** to **EBCDIC** translation, the IBM table **QASCII** in library **QSYS** for **EBCDIC** to **ASCII** translation, or any Alliance table in the product library.

&library (Character, 10)

The name of the library where the translation table resides.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **0000** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMPIFSTRN** in source file **QCLSRC** in the product library gives an example of how to use this API.

FTMIFS400 – Convert an IFS file to an IBM i database file

Description

The **FTMIFS400** API lets you copy an IFS file to an IBM i database file and perform character translation on the IFS file. This is helpful if you are using PGP encryption to encrypt a file, but need to transmit the file as an **EBCDIC** file from an IBM i database file using FTP. You can use this API to convert any IFS file using any translation table. The IFS file is copied to an IBM i database file and translated in the process. If the file does not exist it is created. Note that this API will use the **ASCII** line feed character (hex 0A) to determine the end of a line. Carriage return (hex 0D) and line feed (hex 0A) characters will be converted to spaces.

Calling format

You can call the API like this:

```
Call pgm(ftmifs400) parm(&path &table &tablelib &file &library
&member &reclen &text &return)
```

Parameters

&path (Character, 128)

The fully qualified IFS path and file name. For example,

```
/tmp/myfile.txt
```

&table (Character, 10)

The name of the table you want to use for the translation. You can use the IBM table **QEBCDIC** in library **QSYS** for **ASCII** to **EBCDIC** translation, the IBM table **QASCII** in library **QSYS** for **EBCDIC** to **ASCII** translation, or any Alliance table in the product library.

&tablelib (Character, 10)

The name of the library where the translation table resides.

&file (Character, 10)

The name of the IBM i database file to receive the translated copy of the IFS file. If it does not exist it will be added.

&library (Character, 10)

The name of the library where the file will reside. The library must exist before the API is called.

&member (Character, 10)

The name of the member in the file. If the member does not exist it will be added.

&reclen (Character, 5)

The record length of the file. If the file does not exist the record length will be used to create the file. You must supply leading zeros on this parameter. For example, to create a file with a record length of 80 you would specify **00080** for this value.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **0000** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMPIFS400** in source file **QCLSRC** in the product library gives an example of how to use this API.

FTMIPADDR – Retrieve current 5250 session IP address

Description

The **FTMIPADDR** API lets you retrieve the IP address of the current session. This can be helpful if you want to initiate an FTP session to the interactive user of the current session. This API uses the IBM system API **QDCDEVD** to retrieve the IP address of the current TN5250 session.

Calling format

You can call the API like this:

```
Call pgm(ftmipaddr) parm(&type &ipaddr &text &return)
```

Parameters

&type (Character, 1)

The type of the 5250 session. This value is returned from the API call and can be used to determine if the current session is a value Telnet 5250 session.

&ipaddr (Character, 16)

The IP address of the current session in dotted decimal format.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMPIPADDR`** in source file **`QCLSRC`** in the product library gives an example of how to use this API.

`FTMIPCHG` – Change FTP definition IP address

Description

The **`FTMIPCHG`** API lets you change the IP address and domain name of an FTP definition. This can be helpful if you want to dynamically change an FTP definition from your application programs.

Calling format

You can call the API like this:

```
Call pgm(ftmipchg) parm(&ftpdef &ipaddr &remote &text &return)
```

Parameters

&ftpdef (Character, 10)

The name of the FTP definition you want to change.

&ipaddr (Character, 15)

The new IP address you want to use for the FTP definition. Note that this can be blank if you are specifying a DNS name in the next parameter.

&remote (Character, 24)

The new DNS name for the remote FTP server. If you want to use an IP address in the previous field you must specify the value **`*INTNETADR`** for this field.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **`0000`** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **`SAMIPCHG`** in source file **`QCLSRC`** in the product library gives an example of how to use this API.

`FTMPGPUSER` – Retrieve PGP user ID and pass phrase

Description

The **`FTMPGPUSER`** API lets you retrieve the PGP user ID and pass phrase from an existing FTP definition. This API can be helpful if you are using the PGP commands or APIs to encrypt or decrypt a file and do not want to hard-code the PGP user and pass phrase information in your program.

Calling format

You can call the API like this:

```
Call pgm(ftmpgpuser) parm(&ftpdef &user &pass &text &return)
```

Parameters

&ftpdef (Character, 10)

The name of the FTP definition that contains the PGP user ID and pass phrase.

&user (Character, 64)

The PGP user ID. This value will be returned to your program.

&pass (Character, 32)

The PGP pass phrase. This value will be returned to your program.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program `SAMPGPDEC` in source file `QCLSRC` in the product library gives an example of how to use this API.

`FTMPORT` – Check a remote FTP server port

Description

The `FTMPORT` API lets you check for an active server on the port you specify. You can use this API before you call the Alliance FTP APIs to determine whether or not an FTP server is active on the remote system. This can improve performance. You should only need to use this API if the remote FTP server is know to be unavailable frequently.

Calling format

You can call the API like this:

```
Call pgm(ftmport) parm(&remote &ipaddr &port &text &return)
```

Parameters

&remote (Character, 64)

The DNS name of the remote FTP server. If you want to use an IP address leave this field blank.

&ipaddr (Character, 16)

The IP address of the remote FTP server in dotted decimal format.

&port (Character, 5)

The port number of the remote FTP server. Most FTP servers use port 21 for connections. Specify the value with leading zeros. For example, `00021` .

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `0000` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

`FTMUSER` – Change FTP Definition user and password

Description

The **FTMUSER** API program lets you dynamically change the user ID and password on an FTP definition. The definition must exist before the API is called. The user ID and password are replaced. The password is encrypted if encryption is specified. Note that storing passwords in program source code can lead to security compromises.

Calling format

You can call the API like this:

```
Call pgm(ftmuser) parm(&session &userid &password &text &return)
```

Parameters

&session (Character, 10)

The name of the FTP definition to be changed.

&userid (Character, 15)

The user ID you want to change to. This must be a non-blank user ID for the remote FTP server.

&password (Character, 25)

The password you want to change to. This must be a non-blank password for the remote FTP server.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 4)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is **0000** the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

Sample

The sample program **SAMPUSER** in source file **QCLSRC** in the product library gives an example of how to use this API.

GRPEXEC – Run Group Session

Description

The **GRPEXEC** API provides a simple call interface to run a group session. You pass it the name of a group definition, a text field for the API to return a text description of the session, and a return code. The sessions defined by the group definition will be executed including any substitutions you have defined.

Calling format

You can call the API like this:

```
Call pgm(grpexec) parm(&group &text &return)
```

Parameters

&group (Character, 10)

The name of the group definition to be used for this FTP session.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the previous text field to determine the cause of the failure.

`GRPEXECF` – Run Group Session with File Substitution

Description

The **`GRPEXECF`** API provides a simple call interface to run a group session and lets you specify a local IBM i file name. You pass it the name of a group definition, a file name, a library name containing the file, a text field for the API to return a text description of the session, and a return code. The sessions defined by the group definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Calling format

You can call the API like this:

```
Call pgm(grpexecf) parm(&group &file &library &text &return)
```

Parameters

&group (Character, 10)

The name of the group definition to be used for this FTP session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

`GRPEXECR` – Run Group Session with Local and Remote File Substitution

Description

The `GRPEXECR` API provides a simple call interface to run a group session and lets you specify both a local IBM i file name and the remote file name. You pass it the name of a group definition, a local file and library name, a remote file name, a text field for the API to return a text description of an errors in the session, and a return code. The sessions defined by the group definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Calling format

You can call the API like this:

```
Call pgm(grpexecr) parm(&group &file &library &remote &text &return
```

Parameters

&group (Character, 10)

The name of the group definition to be used for this session.

&file (Character, 10)

The name of the file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file.

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&remote (Character, 64)

The name of the file on the remote system. This name must be compatible with the file system you are sending the file to.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

GRPEXECI – Run Group Session with Local IFS file and Remote File Substitution

Description

The **`GRPEXECI`** API provides a simple call interface to run a group session and lets you specify both a local IBM i IFS file name and the remote file name. You pass it the name of a group definition, a local IFS file name, a remote file name, a text field for the API to return a text description of an errors in the session, and a return code. The session defined by the FTP definition will be executed including any substitutions you have defined. The file you provide on the API line will be used for the local file substitution.

Calling format

You can call the API like this:

```
Call pgm(grpexeci) parm(&group &localfile &remote &text &return)
```

Parameters

&group (Character, 10)

The name of the group definition to be used for this session.

&localfile (Character, 64)

The name of the IFS file you want to use as the local file on the IBM i. The file will be used as the substitution name in the API. You must have enabled substitution on the FTP definition, and must have placed the appropriate substitution parameters in the script file. To send a file in the IFS file system you must specify the fully qualified path. For example if you have a file named “report.txt” in the /user/jsmith IFS directory you would specify this field as:

```
/user/jsmith/report.txt
```

You can send files in the Shared Folder, or **`QDLS`** , file system by qualifying the file name with “QDLS”. For example, if you have a file named “report.txt” in a folder named **`JSMITH`** you would specify the field as:

```
/QDLS/jsmith/report.txt
```

You can use this API to send files in the `QSYS` file system, but you might find it easier to use the `FTMEXECR` API instead. To specify a file in the `QSYS` library system you must fully qualify the name using IFS file naming conventions. For example:

```
/QSYS.LIB/MYLIB.LIB/MYFILE.FILE/MYMEMBER.MBR
```

&library (Character, 10)

The name of the library that contains the file to use in the FTP session.

&remote (Character, 64)

The name of the file on the remote system. This name must be compatible with the file system you are sending the file to.

&text (Character, 80)

A text description describing the success or failure of the conversion text. The API will return text in this field.

&return (Character, 2)

This value is returned by the API to indicate the success or failure of the operation. If the returned value is `00` the request was successful. If the value is anything else the request failed. If the request failed you can use the following text field to determine the cause of the failure.

`API200` – Send email message

Description

This is a generalized API that can be used to send a short message or a long message stored in a database file to an SMTP/Internet recipient. For a short message you can send up to 15 lines of message text. For a long message you specify a database file with the message text. If you use the long message option you must first create the message in a database file before calling this API. This API also lets you specify an attachment for the message.

Note that the call to this API may be successful, but the message may not be delivered to the recipient. There can be many reasons for this including an invalid recipient address, spam controls implemented to Internet Service Providers, or lack of availability of the recipient's mail system. A failure to deliver the message may not be reported back to the sender depending on the reason for the non-delivery.

Call Format

```
CALL PGM(API200) PARM(&TYPE &TOADDR &FROMADDR &REPLYTO &ORGANIZ
        &SMTPHOST &SMTPADDR &SMTPPORT +
        &LOCALHOST &SUBJECT &MSG01 &MSG02 &MSG03 +
        &MSG04 &MSG05 &MSG06 &MSG07 &MSG08 &MSG09 +
        &MSG10 &MSG11 &MSG12 &MSG13 &MSG14 &MSG15 + &FILE &LIBRARY
        &ATCEXTNAM &ATCPATH &ATCNAME &ATCFILE +
        &ATCLIBR &ATCMEMB &RETURN &TEXT)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

Type (Character, 1)

Specify the value of `1` for a short message or the value `2` for a long message stored in a database file. If you specify `1` for a short message you can provide the message in the 15 message lines in the following parameters. If you specify `2` for a long message in a database file you must specify the file, library, and member name that contains the message.

To Address (Character, 128)

The email address of the recipient. This must be a valid SMTP address in the form user@domain.

From address (Character, 128)

The address of the sender. This must be a valid SMTP address, but does not have to the actual address of the user that sends the message. The address must be in the format of user@domain. Please note that most Internet Service Providers will not deliver mail with a sender address with an unregistered domain name. You should also be aware that it is illegal in some jurisdictions to provide a false address on unsolicited email.

Reply-to address (Character, 128)

The address for the reply message. This address can be the same as the From address or can be different. You can specify a return address that is on a different email system and a different domain. See the notes about Internet Service Providers above.

Organization (Character, 128)

Enter the name of your organization. You can provide any text in this field that you like. This would normally be your company name.

SMTP host name (Character, 128)

The name of the SMTP mail server that will be used to deliver this mail. This is normally the name of your mail relay host or ISP mail server and is the domain format. For example: “mail.my-isp.com”. If you do not specify an SMTP host name you must specify an SMTP IP address.

SMTP IP address (Character, 15)

The IP address of your mail relay host or ISP mail server. If you do not specify an SMTP host name you must specify an IP address. The address is in dotted decimal format, for example “206.14.53.98”.

SMTP port (Character, 5)

The port number of the mail server. This is normally `00025` but may be different if you are using an internal mail server or firewall.

Local host (Character, 128)

The name of your local host. This name is sent to the mail server to identify the sending entity. It must be a valid domain name, for example: “as400.acme.com”.

Subject (Character, 50)

The subject for this message. You can supply an text you wish for the subject. The text should not contain any non-printable characters that may cause problems for the recipient’s email application.

Message lines 1 to 15 (Character, 132)

Enter up to 15 lines of message text for a short message. You can leave blank lines in the message, or can leave all lines blank. It is recommended that you use no more than 74 characters in each line in order to stay compatible with most email applications.

File name (Character, 10)

The name of the file containing the message you want to send. The file may be internally or externally described, but must contain only character data. Only the first 132 bytes of the data in each record will be used for the message. Since many email applications expect message line lengths of 74 characters or less it is recommended that you restrict the line length to 74 characters.

Library name (Character, 10)

The name of the library in which the message file resides.

Member (Character, 10)

The name of the member of the message file

Attachment type (Character, 1)

Enter one of the following values for the attachment type. The parameters that follow relating to attachments will be edited based on this parameter. The valid values are:

```
0 = No attachment
1 = QDLS document
2 = IFS file
3 = Database file
4 = Save file
5 = Source file
```

Attachment external name (Character, 64)

Specify the name of the attachment. This can be any valid name for the recipient’s file system. If the recipient is on a DOS computer you should specify this name in the 8.3 format common to DOS. If the recipient is on a Windows 95/98/NT or UNIX system you can specify a long name.

Attachment path (Character, 64)

Specify the attachment path. This is required for attachment types 1 and 2 (QDLS and IFS).

Attachment name (Character, 64)

Specify the name of the attachment in the QDLS or IFS directory. This parameter is required for attachment types 1 and 2.

Attachment file (Character, 10)

If you specified attachment types 3, 4 or 5 you must specify the name of the file that contains the attachment.

Attachment library (Character, 10)

If you specified attachment type 3, 4 or 5 you must specify the name of the library that contains the attachment file.

Attachment member (Character, 10)

If you specified attachment type 3 or 5 you must specify the member name of the attachment. Note that you do not need to specify a member name for Save files and any value you specify for save files will be ignored.

Return Code (Character, 4)

This field is returned as an indication of the success or failure of the call to the API program. If the call was successful and a name is returned, the value of this parameter will be `0000`. Any other value indicates an error in processing the request. If an error occurs there may be information in the following field describing the error.

Text (Character, 80)

This field contains text that describes the success or failure of the API. The field may be blank if the API call was successful.

Problem Analysis

If an error occurs in the API an error indicator is returned in the four byte return code. Any return code other than `0000` indicates an error condition. Generally if there is an error condition there will be text in the Text field that describes the error. You can also view a description of the error by viewing the associated error in the MSG001 message file in the application library. The message ID will be "TMS" plus the return code. For example, if the return code is "1646" you can view the error message associates with this return code with the following command:

```
dspmsgd range(tms1646) msgf(msg001)
```

You can also enable the NetMail application trace flag in data area TMICFT. When this flag is enabled the API will generate additional diagnostic information to the job log of the calling program. To enable application tracing change the data area TMICFT to the value `Y`:

```
chgdtaara tmicft value("Y")
```

After calling the API display the job log and review the messages for diagnostic information. You can use the **DSPJOBLOG** command to view the job log. If the job was submitted to batch you can use the **WRKSBMJOB** or **WRKUSRJOB** commands to view the printed job log. Note that you may have to change the job description used to submit the job to produce the printed job log. Remember to change the data area TMICFT to **N** when you are finished to avoid unnecessary printed output and the performance impact of additional logging.

If you think the error may be occurring in the SMTP interface or with the connection to your SMTP mail server, you can turn on the application logging function and review the resulting log in file **TMLOGA** . The application logging function is turned on from the administration menu. After turning on the logging function you can view the log using the **DSPPFM** command. There may be diagnostic information in this file that describe the problem. All of the responses from the SMTP mail server are logging in this file. Note that you will want to turn off application logging as soon as you are finished to avoid accumulating logging records and to improve performance. You can clear the log with the **CLRPFM** command:

```
clrpfm tmloga
```

API201 – Send email message with Spool file

Description

This is a generalized API that can be used to send a short message or a long message stored in a database file to an SMTP/Internet recipient with a spool file attachment. For a short message you can send up to 15 lines of message text. For a long message you specify a database file with the message text. If you use the long message option you must first create the message in a database file before calling this API. This API also lets you specify an attachment for the message.

Note that the call to this API may be successful, but the message may not be delivered to the recipient. There can be many reasons for this including an invalid recipient address, spam controls implemented to Internet Service Providers, or lack of availability of the recipient's mail system. A failure to deliver the message may not be reported back to the sender depending on the reason for the non-delivery.

Call Format

```
CALL PGM(API201) PARM(&TYPE &TOADDR &FROMADDR &REPLYTO +

         &ORGANIZ &SMTPHOST &SMTPADDR &SMTPPORT +
         &LOCALHOST &SUBJECT &MSG01 &MSG02 &MSG03 +
         &MSG04 &MSG05 &MSG06 &MSG07 &MSG08 &MSG09 +
         &MSG10 &MSG11 &MSG12 &MSG13 &MSG14 &MSG15 +
         &FILE &LIBRARY &MEMBER &ATCEXTNAM +
         &SPLFNAME &SPLJOBNAM &SPLJOBUSR &SPLJOBNO +
         &SPLFNUM &RETURN &TEXT)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

Type (Character, 1)

Specify the value of `1` for a short message or the value `2` for a long message stored in a database file. If you specify `1` for a short message you can provide the message in the 15 message lines in the following parameters. If you specify `2` for a long message in a database file you must specify the file, library, and member name that contains the message.

To Address (Character, 128)

The email address of the recipient. This must be a valid SMTP address in the form user@domain.

From address (Character, 128)

The address of the sender. This must be a valid SMTP address, but does not have to the actual address of the user that sends the message. The address must be in the format of user@domain. Please note that most Internet Service Providers will not deliver mail with a sender address with an unregistered domain name. You should also be aware that it is illegal in some jurisdictions to provide a false address on unsolicited email.

Reply-to address (Character, 128)

The address for the reply message. This address can be the same as the From address or can be different. You can specify a return address that is on a different email system and a different domain. See the notes about Internet Service Providers above.

Organization (Character, 128)

Enter the name of your organization. You can provide any text in this field that you like. This would normally be your company name.

SMTP host name (Character, 128)

The name of the SMTP mail server that will be used to deliver this mail. This is normally the name of your mail relay host or ISP mail server and is the domain format. For example: “mail.my-isp.com”. If you do not specify an SMTP host name you must specify an SMTP IP address.

SMTP IP address (Character, 15)

The IP address of your mail relay host or ISP mail server. If you do not specify an SMTP host name you must specify an IP address. The address is in dotted decimal format, for example “206.14.53.98”.

SMTP port (Character, 5)

The port number of the mail server. This is normally `00025` but may be different if you are using an internal mail server or firewall.

Local host (Character, 128)

The name of your local host. This name is sent to the mail server to identify the sending entity. It must be a valid domain name, for example: “as400.acme.com”.

Subject (Character, 50)

The subject for this message. You can supply an text you wish for the subject. The text should not contain any non-printable characters that may cause problems for the recipient's email application.

Message lines 1 to 15 (Character, 132)

Enter up to 15 lines of message text for a short message. You can leave blank lines in the message, or can leave all lines blank. It is recommended that you use no more than 74 characters in each line in order to stay compatible with most email applications.

File name (Character, 10)

The name of the file containing the message you want to send. The file may be internally or externally described, but must contain only character data. Only the first 132 bytes of the data in each record will be used for the message. Since many email applications expect message line lengths of 74 characters or less it is recommended that you restrict the line length to 74 characters.

Library name (Character, 10)

The name of the library in which the message file resides.

Member (Character, 10)

The name of the member of the message file.

Attachment external name (Character, 64)

Specify the name of the attachment. This can be any valid name for the recipient's file system. If the recipient is on a DOS computer you should specify this name in the 8.3 format common to DOS. If the recipient is on a Windows 95/98/NT or UNIX system you can specify a long name.

Spool file name (Character, 10)

Specify the name of the spool file you want to attach to this message. The spool file must exist at the time the API is called.

Job name (Character, 10)

Specify the name of the job that created the spool file.

User name (Character, 10)

Specify the name of the user (user profile) that created the spool file.

Job number (Character, 6)

Specify the job number for the job that created the spool file.

Spool file number (Character, 6)

Specify the number of the spool file. This must be 4 digits in the range of `0001` to `9999`, or the special values of `*LAST` or `*ONLY`.

Return Code (Character, 4)

This field is returned as an indication of the success or failure of the call to the API program. If the call was successful and a name is returned, the value of this parameter will be `0000`. Any other value indicates an error in processing the request. If an error occurs there may be information in the following field describing the error.

Text (Character, 80)

This field contains text that describes the success or failure of the API. The field may be blank if the API call was successful.

Problem Analysis

If an error occurs in the API an error indicator is returned in the four byte return code. Any return code other than `0000` indicates an error condition. Generally if there is an error condition there will be text in the Text field that describes the error. You can also view a description of the error by viewing the associated error in the MSG001 message file in the application library. The message ID will be “TMS” plus the return code. For example, if the return code is “1646” you can view the error message associates with this return code with the following command:

```
dspmsgd range(tms1646) msgf(msg001)
```

You can also enable the NetMail application trace flag in data area TMICFT. When this flag is enabled the API will generate additional diagnostic information to the job log of the calling program. To enable application tracing change the data area TMICFT to the value `Y` :

```
chgdtaara tmicft value(“Y”)
```

After calling the API display the job log and review the messages for diagnostic information. You can use the `DSPJOBLOG` command to view the job log. If the job was submitted to batch you can use the `WRKSBMJOB` or `WRKUSRJOB` commands to view the printed job log. Note that you may have to change the job description used to submit the job to produce the printed job log. Remember to change the data area TMICFT to `N` when you are finished to avoid unnecessary printed output and the performance impact of additional logging.

If you think the error may be occurring in the SMTP interface or with the connection to your SMTP mail server, you can turn on the application logging function and review the resulting log in file `TMLOGA` . The application logging function is turned on from the administration menu. After turning on the logging function you can view the log using the `DSPPFM` command. There may be diagnostic information in this file that describe the problem. All of the responses from the SMTP mail server are logging in this file. Note that you will want to turn off application logging as soon as you are finished to avoid accumulating logging records and to improve performance. You can clear the log with the `CLRPFM` command:

```
clrpfm tmloga
```

`API216` – Send File Message or Short Message to multiple recipients

Description

This is a generalized API that can be used to send a short message or a long message stored in a database file to one or more SMTP/Internet recipients. For a short message you can send up to 15 lines of message text. For a long message you specify a database file with the message text. If you use the long message option you must first create the message in a database file before calling this API. This API also lets you specify an attachment for the message. This API is similar to API200 with the exception that is allows you to specify multiple To, CC and BCC recipients.

Note that the call to this API may be successful, but the message may not be delivered to the recipient. There can be many reasons for this including an invalid recipient address, spam controls implemented to Internet Service Providers, or lack of availability of the recipient's mail system. A failure to deliver the message may not be reported back to the sender depending on the reason for the non-delivery.

Call Format

```
CALL PGM(API216) PARM(&TYPE &TOADDR &CCADDR &BCADDR &FROMADDR
&REPLYTO &ORGANIZ &SMTPHOST &SMTPADDR &SMTPPORT +
         &LOCALHOST &SUBJECT &MSG01 &MSG02 &MSG03 +
         &MSG04 &MSG05 &MSG06 &MSG07 &MSG08 &MSG09 +
         &MSG10 &MSG11 &MSG12 &MSG13 &MSG14 &MSG15 +
         &FILE &LIBRARY &MEMBER &ATCTYPE +
         &ATCEXTNAM &ATCPATH &ATCNAME &ATCFILE +
         &ATCLIBR &ATCMEMB &RETURN &TEXT)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

Type (Character, 1)

Specify the value of `1` for a short message or the value `2` for a long message stored in a database file. If you specify `1` for a short message you can provide the message in the 15 message lines in the following parameters. If you specify `2` for a long message in a database file you must specify the file, library, and member name that contains the message.

To Address (Character, 9999)

The email addresses of the recipient. This must be one or more valid SMTP addresses in the form user@domain. If there are multiple addresses you should separate them with a comma. For example:

```
User1@acme.com, User2@acme.com, User3@acme.com
```

CC Addresses (Character, 9999)

The email addresses of the CC recipients. This must be one or more valid SMTP addresses in the form user@domain. If there are multiple addresses you should separate them with a comma. For example:

```
User1@acme.com, User2@acme.com, User3@acme.com
```

BCC Addresses (Character, 9999)

The email addresses of the BCC (Blind Carbon Copy) recipients. This must be one or more valid SMTP addresses in the form user@domain. If there are multiple addresses you should separate them with a comma. For example:

```
User1@acme.com, User2@acme.com, User3@acme.com
```

From address (Character, 128)

The address of the sender. This must be a valid SMTP address, but does not have to the actual address of the user that sends the message. The address must be in the format of user@domain. Please note that most Internet Service Providers will not deliver mail with a sender address with an unregistered domain name. You should also be aware that it is illegal in some jurisdictions to provide a false address on unsolicited email.

Reply-to address (Character, 128)

The address for the reply message. This address can be the same as the From address or can be different. You can specify a return address that is on a different email system and a different domain. See the notes about Internet Service Providers above.

Organization (Character, 128)

Enter the name of your organization. You can provide any text in this field that you like. This would normally be your company name.

SMTP host name (Character, 128)

The name of the SMTP mail server that will be used to deliver this mail. This is normally the name of your mail relay host or ISP mail server and is the domain format. For example: “mail.my-isp.com”. If you do not specify an SMTP host name you must specify an SMTP IP address.

SMTP IP address (Character, 15)

The IP address of your mail relay host or ISP mail server. If you do not specify an SMTP host name you must specify an IP address. The address is in dotted decimal format, for example “206.14.53.98”.

SMTP port (Character, 5)

The port number of the mail server. This is normally `00025` but may be different if you are using an internal mail server or firewall.

Local host (Character, 128)

The name of your local host. This name is sent to the mail server to identify the sending entity. It must be a valid domain name, for example: “as400.acme.com”.

Subject (Character, 50)

The subject for this message. You can supply any text you wish for the subject. The text should not contain any non-printable characters that may cause problems for the recipient's email application.

Message lines 1 to 15 (Character, 132)

Enter up to 15 lines of message text for a short message. You can leave blank lines in the message, or can leave all lines blank. It is recommended that you use no more than 74 characters in each line in order to stay compatible with most email applications.

File name (Character, 10)

The name of the file containing the message you want to send. The file may be internally or externally described, but must contain only character data. Only the first 132 bytes of the data in each record will be used for the message. Since many email applications expect message line lengths of 74 characters or less it is recommended that you restrict the line length to 74 characters.

Library name (Character, 10)

The name of the library in which the message file resides.

Member (Character, 10)

The name of the member of the message file

Attachment type (Character, 1)

Enter one of the following values for the attachment type. The parameters that follow relating to attachments will be edited based on this parameter. The valid values are:

```
0 = No attachment
1 = QDLS document
2 = IFS file
3 = Database file
4 = Save file
5 = Source file
```

Attachment external name (Character, 64)

Specify the name of the attachment. This can be any valid name for the recipient's file system. If the recipient is on a DOS computer you should specify this name in the 8.3 format common to DOS. If the recipient is on a Windows 95/98/NT or UNIX system you can specify a long name.

Attachment path (Character, 64)

Specify the attachment path. This is required for attachment types 1 and 2 (QDLS and IFS).

Attachment name (Character, 64)

Specify the name of the attachment in the QDLS or IFS directory. This parameter is required for attachment types 1 and 2.

Attachment file (Character, 10)

If you specified attachment types 3, 4 or 5 you must specify the name of the file that contains the attachment.

Attachment library (Character, 10)

If you specified attachment type 3, 4 or 5 you must specify the name of the library that contains the attachment file.

Attachment member (Character, 10)

If you specified attachment type 3 or 5 you must specify the member name of the attachment. Note that you do not need to specify a member name for Save files and any value you specify for save files will be ignored.

Return Code (Character, 4)

This field is returned as an indication of the success or failure of the call to the API program. If the call was successful and a name is returned, the value of this parameter will be `0000` . Any other value indicates an error in processing the request. If an error occurs there may be information in the following field describing the error.

Text (Character, 80)

This field contains text that describes the success or failure of the API. The field may be blank if the API call was successful.

Problem Analysis

If an error occurs in the API an error indicator is returned in the four byte return code. Any return code other than `0000` indicates an error condition. Generally if there is an error condition there will be text in the Text field that describes the error. You can also view a description of the error by viewing the associated error in the MSG001 message file in the application library. The message ID will be “TMS” plus the return code. For example, if the return code is `1646` you can view the error message associates with this return code with the following command:

```
dspmsgd range(tms1646) msgf(msg001)
```

You can also enable the NetMail application trace flag in data area TMICFT. When this flag is enabled the API will generate additional diagnostic information to the job log of the calling program. To enable application tracing change the data area TMICFT to the value `Y` :

```
chgdtaara tmicft value(“Y”)
```

After calling the API display the job log and review the messages for diagnostic information. You can use the `DSPJOBLOG` command to view the job log. If the job was submitted to batch you can use the `WRKSBMJOB` or `WRKUSRJOB` commands to view the printed job log. Note that you may have to change the job description used to submit the job to produce the printed job log. Remember to change the data area TMICFT to `N` when you are finished to avoid unnecessary printed output and the performance impact of additional logging.

If you think the error may be occurring in the SMTP interface or with the connection to your SMTP mail server, you can turn on the application logging function and review the resulting log in file TMLOGA. The application logging function is turned on from the administration menu. After turning on the logging function you can view the log using the `DSPPFM` command. There

may be diagnostic information in this file that describe the problem. All of the responses from the SMTP mail server are logging in this file. Note that you will want to turn off application logging as soon as you are finished to avoid accumulating logging records and to improve performance. You can clear the log with the **CLRPFM** command:

```
clrpfm tmloga
```

API218 – Initialize attachment list

Description

This API is used to create and initialize an list of attachments. You can use this API before calling selected email APIs to add multiple attachments to an email message. After calling **API218** to initialize the attachment list you can use **API219** to add attachments. When you are finished with the list of attachments you can call **API220** to destroy the list.

Call Format

```
CALL PGM(API218) PARM(&RETURN &TEXT)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

Return Code (Character, 4)

This field is returned as an indication of the success or failure of the call to the API program. If the call was successful and a name is returned, the value of this parameter will be **0000**. Any other value indicates an error in processing the request. If an error occurs there may be information in the following field describing the error.

Text (Character, 80)

This field contains text that describes the success or failure of the API. The field may be blank if the API call was successful.

Problem Analysis

If an error occurs in the API an error indicator is returned in the four byte return code. Any return code other than **0000** indicates an error condition. Generally if there is an error condition there will be text in the Text field that describes the error. You can also view a description of the error by viewing the associated error in the MSG001 message file in the application library. The message ID will be “TMS” plus the return code. For example, if the return code is **1646** you can view the error message associates with this return code with the following command:

```
dspmsgd range(tms1646) msgf(msg001)
```

You can also enable the NetMail application trace flag in data area TMICFT. When this flag is enabled the API will generate additional diagnostic information to the job log of the calling program. To enable application tracing change the data area TMICFT to the value **`Y`**:

```
chgdtaara tmicft value("Y")
```

After calling the API display the job log and review the messages for diagnostic information. You can use the **`DSPJOBLOG`** command to view the job log. If the job was submitted to batch you can use the **`WRKSBMJOB`** or **`WRKUSRJOB`** commands to view the printed job log. Note that you may have to change the job description used to submit the job to produce the printed job log. Remember to change the data area TMICFT to **`N`** when you are finished to avoid unnecessary printed output and the performance impact of additional logging.

If you think the error may be occurring in the SMTP interface or with the connection to your SMTP mail server, you can turn on the application logging function and review the resulting log in file TMLOGA. The application logging function is turned on from the administration menu. After turning on the logging function you can view the log using the **`DSPPFM`** command. There may be diagnostic information in this file that describe the problem. All of the responses from the SMTP mail server are logging in this file. Note that you will want to turn off application logging as soon as you are finished to avoid accumulating logging records and to improve performance. You can clear the log with the **`CLRPFM`** command:

```
clrpfm tmloga
```

`API219` – Add attachment to a list

Description

This API is used to add an attachment to a list. You must first call **`API218`** to initialize the list before calling this API. Once you have called **`API218`** to initialize the list you can call this API as many times as you like to add attachments.

Call Format

```
CALL PGM(API219) PARM(&ACCOUNT &ATCTYPE &ATCEXTNAM &ATCPATH +
        &ATCNAME &ATCFILE &ATCLIBR &ATCMEMB +
        &RETURN &TEXT)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

Account (Character, 10)

Enter the user name for this attachment. This is usually the name of the user profile that is calling the API. If you leave this parameter blank the user of the job that calls the API will be used.

Attachment type (Character, 1)

Enter one of the following values for the attachment type. The parameters that follow relating to attachments will be edited based on this parameter. The valid values are:

```
1 = QDLS document
2 = IFS file
3 = Database file
4 = Save file
5 = Source file
```

Attachment external name (Character, 64)

Specify the name of the attachment. This can be any valid name for the recipient's file system. If the recipient is on a DOS computer you should specify this name in the 8.3 format common to DOS. If the recipient is on a Windows 95/98/NT or UNIX system you can specify a long name.

Attachment path (Character, 64)

Specify the attachment path. This is required for attachment types 1 and 2 (QDLS and IFS).

Attachment name (Character, 64)

Specify the name of the attachment in the QDLS or IFS directory. This parameter is required for attachment types 1 and 2.

Attachment file (Character, 10)

If you specified attachment types 3, 4 or 5 you must specify the name of the file that contains the attachment.

Attachment library (Character, 10)

If you specified attachment type 3, 4 or 5 you must specify the name of the library that contains the attachment file.

Attachment member (Character, 10)

If you specified attachment type 3 or 5 you must specify the member name of the attachment. Note that you do not need to specify a member name for Save files and any value you specify for save files will be ignored.

Return Code (Character, 4)

This field is returned as an indication of the success or failure of the call to the API program. If the call was successful and a name is returned, the value of this parameter will be `0000`. Any other value indicates an error in processing the request. If an error occurs there may be information in the following field describing the error.

Text (Character, 80)

This field contains text that describes the success or failure of the API. The field may be blank if the API call was successful.

Problem Analysis

If an error occurs in the API an error indicator is returned in the four byte return code. Any return code other than `0000` indicates an error condition. Generally if there is an error condition there will be text in the Text field that describes the error. You can also view a description of the error by viewing the associated error in the MSG001 message file in the application library. The message ID will be “TMS” plus the return code. For example, if the return code is `1646` you can view the error message associates with this return code with the following command:

```
dspmsgd range(tms1646) msgf(msg001)
```

You can also enable the NetMail application trace flag in data area TMICFT. When this flag is enabled the API will generate additional diagnostic information to the job log of the calling program. To enable application tracing change the data area TMICFT to the value `Y` :

```
chgdtaara tmicft value(“Y”)
```

After calling the API display the job log and review the messages for diagnostic information. You can use the `DSPJOBLOG` command to view the job log. If the job was submitted to batch you can use the `WRKSBMJOB` or `WRKUSRJOB` commands to view the printed job log. Note that you may have to change the job description used to submit the job to produce the printed job log. Remember to change the data area TMICFT to `N` when you are finished to avoid unnecessary printed output and the performance impact of additional logging.

If you think the error may be occurring in the SMTP interface or with the connection to your SMTP mail server, you can turn on the application logging function and review the resulting log in file TMLOGA. The application logging function is turned on from the administration menu. After turning on the logging function you can view the log using the DSPPFM command. There may be diagnostic information in this file that describe the problem. All of the responses from the SMTP mail server are logging in this file. Note that you will want to turn off application logging as soon as you are finished to avoid accumulating logging records and to improve performance. You can clear the log with the `CLRPFM` command:

```
clrpfm tmloga
```

`API220` – Destroy attachment list

Description

This API is used to destroy an attachment list that you created with `API217` and `API218` . You should call this API when you are finished sending email messages with attachments. Failure to destroy the attachment list may lead to unexpected attachments for email messages you send with other APIs.

Call Format

```
CALL PGM(API220)
```

Parameters

The following parameters are passed to the API. They must be specified in the following order and each field passed to the API must have the type and length specified.

There are no parameters for this API.

ZIP – Compress an IFS File

Description

> ***NOTE:*** For new file compression projects you should use the **FTMZIP** command which provides better file handling options and AES encryption. This callable API will be maintained but not enhanced.

The **ZIP** program compresses one or more files in the IFS directory into a standard Zip archive file. The resulting file is compatible with PC-based programs like PKUnzip and WinZip. You can directly call the **ZIP** program to compress your files. If you want to compress an IBM i database file you must first copy the file to an IFS directory. You can use the **CPYTOSTMF** command to do this. The Zip program can perform **EBCDIC** to **ASCII** conversion for you if you like. If you copy the file using **CPYTOSTMF** you have the option to covert to **ASCII** during the copy operation. If you convert the file with **CPYTOSTMF** be sure to not convert it again with the Zip "-a" option.

Calling format

You can call the **ZIP** API like this:

```
Call pgm(zip) parm(&options &zipfile &file1 &file2 …)
```

Parameters

&options (Character, 32)

A list of option flags that direct various operations of the zip program. Some common options are:

-f freshen: only changed files

-d delete entries in zipfile

-r recurse into directories

-0 store only

-1 compress faster

-q quiet operation

-c add one-line comments

-@ read names from stdin

-x exclude the following names

-a translate to **ASCII**

-F fix zipfile (-FF try harder)

-A adjust self-extracting exe

-T test zipfile integrity

-R PKZIP recursion (see manual)

-h show this help

-u update: only changed or new files

-m move into zipfile (delete files)

-j junk (don’t record) directory names

-l convert LF to CR LF (-ll CR LF to LF)

-9 compress better

-v verbose operation/print version info

-z add zipfile comment

-o make zipfile as old as latest entry

-i include only the following names

-D do not add directory entries

-J junk zipfile prefix (unzipsfx)

-X eXclude eXtra file attributes

-n don’t compress these suffixes

Please note that not all Zip options may be supported on the IBM i at the current time. The options a, v, q, and j are the most commonly used and are supported. If you are calling the API from a program you should null terminate the character variable (see the example below).

&zipfile (Character, 256)

The name of the archive file you want to create. This should normally be a file name with a “.zip” extension. Provide the name as a fully qualified IFS file name:

```
/tmp/myfile.zip
```

&file1 &file2 … (Character, 256)

One or more file files that you want to add to the zip archive. These must be files in an IFS directory. If you want to add an IBM i database file to a zip archive you must first copy it to an IFS directory using the **CPYTOSTMF** command. You can add more than one file to the archive, just name the files in the order you want them added.

Examples

You can call the Zip utility from the IBM i command line. This is an example of how to call Zip:

```
Call zip parm('-qj' '/tmp/MyArchive.zip' '/tmp/File1.csv'
'/tmp/File2.txt')
```

This example calls Zip in quiet mode and removes the directory paths, creates a zip file named MyArchive.zip in the /tmp IFS directory, and adds the files File1.csv and File2.txt in IFS directory /tmp to the archive. No **ASCII** conversion is performed.

You can also call Zip from your application programs. This is an example of how to call Zip from a CL program:

```
DCL        VAR(&OPTIONS) TYPE(*CHAR) LEN(10)
DCL        VAR(&ZIPFILE) TYPE(*CHAR) LEN(128)
DCL        VAR(&FILE1) TYPE(*CHAR) LEN(128)
DCL        VAR(&FILE2) TYPE(*CHAR) LEN(128)
DCL        VAR(&NULL) TYPE(*CHAR) LEN(1) VALUE(X'00')

CHGVAR     VAR(&OPTIONS) VALUE('-aqj' *TCAT &NULL)
CHGVAR     VAR(&ZIPFILE) VALUE('/tmp/MyArchive.zip' +
             *TCAT &NULL)
CHGVAR     VAR(&FILE1) VALUE('/tmp/File1.csv' *TCAT &NULL)
CHGVAR     VAR(&FILE2) VALUE('/tmp/File2.txt' *TCAT &NULL)

CALL       PGM(ZIP) PARM(&OPTIONS &ZIPFILE &FILE1 &FILE2)
```

This example is similar to the first with the exception that **ASCII** conversion is performed with the “-a” option.

UNZIP – Decompress an IFS File

Description

> ***NOTE:*** For new file compression projects you should use the **FTMUNZIP** command which provides better file handling options and AES decryption. This callable API will be maintained but not enhanced.

The **UNZIP** program decompresses one or more files in the IFS directory into stream files. The resulting files can be copied to an IBM i library using the Copy From Stream File (**CPYFRMSTMF**) command, or Copy From Import File (**CPYFRMIMPF**) command.

Calling format

You can call the **UNZIP** API like this:

```
Call pgm(unzip) parm('-o' '/tmp/file.zip')
```

Parameters

&options (Character, 32)

A list of option flags that direct various operations of the unzip program. Some common options are:

-p extract files to pipe, no messages

-l list files (short format)

-f freshen existing files, create none

-t test compressed archive data

-u update files, create if necessary

-z display archive comment

-x exclude files that follow (in xlist)

-d extract files into exdir

modifiers:

-q quiet mode (-qq => quieter)

-n never overwrite existing files

-a auto-convert any text files

-aa treat ALL files as text

-o overwrite files WITHOUT prompting

-j junk paths (do not make directories)

-v be verbose/print version info

-C match filenames case-insensitively

-L make (some) names lowercase

-X restore UID/GID info

-V retain VMS version numbers

-M pipe through “more” pager

Please note that not all Unzip options may be supported on the IBM i at the current time. If you are calling the API from a program you should null terminate the character variable (see the example below).

Examples

The CL program **`SAMUNZIP`** in source file **`QCLSRC`** in the Alliance library provides an example of using the Unzip API program.

Sample Programs

Sample programs are provided in the **`QCLSRC`** and **`QRPGSRC`** files in the product library. You may use these source files as a template or guide for your own programs. Be sure to store your own programs in a library other than the FTP Manager product library. This library will be

refreshed periodically and you will lose any source you store in these source members. This does not apply to FTP script files in SCRIPT which are always restored as a part of a release upgrade.

Chapter 5: Bindable Application Program Interfaces

Overview

The FTP Manager bindable application program interfaces provide an easy way to use various utility functions available in Alliance FTP Manager service programs. These APIs are located in service programs in the Alliance product library and may be bound to any ILE application. They provide return codes that let you know the status of your request. A zero value in the return code indicates success with the API call.

FromBase64 – Convert from Base64 to Binary

This procedure converts a Base64 encoded string to binary value.

Procedure definition:

```
void FromBase64(char *string, long *length, char *response,
long *rlength, long *error)
```

ILE RPG definition:

```
d String          s             256a   inz(' ')
d Length          s               8b 0 inz(0)
d Response        s             512a   inz(' ')
d RtnLength       s               8b 0 inz(0)
d Error           s               8b 0 inz(0)

c                   Callb(d)   'FromBase64'
c                   Parm                    String
c                   Parm                    Length
c                   Parm                    Response
c                   Parm                    RtnLength
c                   Parm                    Error
```

Parameter definition:

String (Alpha, 256)

The input Base64 encoded string to be converted. The length of the Base64 encoded string can be up to 32767 bytes.

Length (Binary, 8)

The length of the Base64 encoded string.

Response (Alpha, 256)

The variable to receive the Base64 decoded value. The size of this variable must be large enough to hold the decoded data.

RtnLength (Binary, 8)

The length of the converted data. This value will be returned to your program.

Error (Binary, 8)

The return code from the procedure. The value of `0` (zero) indicates a successful call to the API. Any other value indicates an error.

Service program

The FromBase64 procedure is contained in the service program **`ANBASE64`**.

Sample code

The sample ILE RPG program **`SAMPB64A`** is located in source file **`QRPGLESRC`** in the product library.

Error codes

2598 – The Base64 character data contains an invalid value. Only upper case alphabetic, lower case alphabetic, numbers, and the special values “+” and “/” are allowed in Base64 encoded data.

2599 – A Base64 coded character data continuation character was found in the wrong position. The “=” character is only allowed in the second or third position of a terminating Base64 character string. It was found in the first position.

ToBase64 – Convert from Binary to Base64

This procedure converts a binary string to Base64 encoded value.

Procedure definition:

```
void ToBase64(char *string, long *length, char *response,
long *rlength, long *error)
```

ILE RPG definition:

```
d String          s            256a   inz(' ')
d Length          s              8b 0 inz(0)
d Response        s            512a   inz(' ')
d RtnLength       s              8b 0 inz(0)
d Error           s              8b 0 inz(0)

c                   Callb(d)   'ToBase64'
c                   Parm                    String
c                   Parm                    Length
c                   Parm                    Response
c                   Parm                    RtnLength
c                   Parm                    Error
```

Parameter definition:

String (Alpha, 256)

The input data to be encoded to Base64 format. The length of the string can be up to 24571 bytes.

Length (Binary, 8)

The length of the binary string to convert.

Response (Alpha, 256)

The variable to receive the Base64 encoded value. The size of this variable must be large enough to hold the decoded data. For every 3 characters of input data, 4 characters of Base64 encoded data will be produced.

RtnLength (Binary, 8)

The length of the converted data. This value will be returned to your program.

Error (Binary, 8)

The return code from the procedure. The value of `0` (zero) indicates a successful call to the API. Any other value indicates an error.

Service program

The ToBase64 procedure is contained in the service program **`ANBASE64`**.

Sample code

The sample ILE RPG program **`SAMPB64A`** is located in source file **`QRPGLESRC`** in the product library.

Error codes

`2600` – The input length of the data to be converted must not exceed 24571 bytes. The input data is too long.

`2597` – The input data value is incorrect. A character was encountered with a decimal value greater than 255.